https://www.h2kinfosys.com/blog/python-data-analyst-interview-questions-answers/

What is Parsing?
Interview Answer
Parsing means reading structured data like JSON, XML, or HTML and converting it into a format that Python can understand and process.

---

2. What is JSON?
JSON (JavaScript Object Notation) is a lightweight structured data format used for storing and exchanging data using key-value pairs.
3. What is XML?
XML (eXtensible Markup Language) is a tag-based structured data format used to store and transport data.
5. What is BeautifulSoup?
BeautifulSoup is a Python library used for parsing HTML and XML documents and extracting data from webpages.

---

6. What is Web Scraping?
Web scraping means extracting data automatically from websites using Python tools like BeautifulSoup and Selenium.

---

7. Why BeautifulSoup is Used?
Used for:
extracting webpage data
parsing HTML/XML
finding tags/elements
extracting:
text
tables
links
job data
. What is HTML Parsing?
HTML parsing means reading webpage HTML structure and extracting required information from tags.

---

10. Important JSON Functions

Function	Purpose
`json.loads()`	Parse JSON string
`json.load()`	Read JSON file
`json.dumps()`	Convert Python → JSON

---

11. Difference Between `load()` and `loads()`
Very Common Interview Question

Function	Use
`json.load()`	Reads JSON from file

Function	Use
`json.loads()`	Reads JSON from string

---

12. Important BeautifulSoup Functions

Function	Purpose
`find()`	Find first tag
`find_all()`	Find all matching tags
`.text`	Extract text

---

13. Example Interview Coding Question
Q: How do you parse JSON in Python?

```
import json

data='{"name":"Shama"}'

parsed=json.loads(data)

print(parsed["name"])
```

---

14. XML Parsing Example

```
from bs4 import BeautifulSoup

xml="""
<employee>
  <name>Shama</name>
</employee>
"""

soup=BeautifulSoup(xml,"xml")

print(soup.find("name").text)
```

---

15. BeautifulSoup Example

```
from bs4 import BeautifulSoup

html="<h1>Hello</h1>"

soup=BeautifulSoup(html,"html.parser")

print(soup.h1.text)
```

. Complete Example — BeautifulSoup with HTML
This example extracts:
heading
paragraph
link
from HTML.

HTML Example

```python
from bs4 import BeautifulSoup

html = """
<html>
   <body>

      <h1>Welcome</h1>

      <p>This is BeautifulSoup example</p>

      <a href="https://google.com">Google</a>

   </body>
</html>
"""

# Parse HTML
soup = BeautifulSoup(html, "html.parser")

# Extract heading
heading = soup.find("h1").text

# Extract paragraph
paragraph = soup.find("p").text

# Extract link
link = soup.find("a")["href"]

print("Heading:", heading)
print("Paragraph:", paragraph)
print("Link:", link)
```

2. Complete Example — BeautifulSoup with XML
This example extracts:
employee names
roles
from XML.

XML Example

Python

```python
from bs4 import BeautifulSoup

xml = """
<employees>

   <employee>
      <name>Shama</name>
      <role>Data Analyst</role>
   </employee>

   <employee>
      <name>Rahul</name>
      <role>Python Developer</role>
   </employee>

</employees>
"""

# Parse XML
soup = BeautifulSoup(xml, "xml")

# Find all employee tags
employees = soup.find_all("employee")

for emp in employees:

    name = emp.find("name").text

    role = emp.find("role").text

    print(name, "-", role)
```

Ask ChatGPT

Most Important Functions Used

Function	Meaning
`BeautifulSoup()`	Parse document
`find()`	Find first tag
`find_all()`	Find all matching tags
`.text`	Extract text
`["href"]`	Extract attribute

Ask ChatGPT

Interview Explanation

You can say:

> "BeautifulSoup is used to parse HTML and XML documents. Using methods like `find()` and `find_all()`, we can extract tags, text, and attributes from structured data."

PYTHON

Q1. How do you remove special characters from a column?

df['Name'] = df['Name'].replace(r'[^a-zA-Z0-9]', '', regex=True)

Keeps letters, digits and spaces; removes other characters.

Q2. How do you remove leading and trailing spaces?
df['Name'] = df['Name'].str.strip()
Q3. How do you convert a column into lowercase?
df['Name'] = df['Name'].str.lower()
Q4. How do you convert a column into uppercase?
df['Name'] = df['Name'].str.upper()
Q5. How do you replace missing values with 0?
df['Salary'] = df['Salary'].fillna(0)
Use 0 only when it makes sense for the data.
Q8. How do you replace multiple values in a column?
df['City'] = df['City'].replace({'Pune City': 'Pune', 'Bombay': 'Mumbai'})
Q9. Retrieve employees whose salary is greater than 50,000.
df[df['Salary'] > 50000]
Q10. Retrieve employees from Pune.
df[df['City'] == 'Pune']
Q11. Retrieve Pune employees whose salary is greater than 50,000.
df[(df['City'] == 'Pune') & (df['Salary'] > 50000)]
Use & for AND and | for OR. Put each condition in parentheses.
Q12. Retrieve employees from Pune or Mumbai.
df[df['City'].isin(['Pune', 'Mumbai'])]
Q13. Sort salary in descending order.
df.sort_values(by='Salary', ascending=False)
Q14. Find the top five highest-paid employees.
df.nlargest(5, 'Salary')
It will return the whole DataFrame (all columns) for the top 5 highest-paid employees, not just the Salary column.

If you want only the Salary column: df.nlargest(5, 'Salary')['Salary']
Q15. Calculate average salary city-wise.
df.groupby('City')['Salary'].mean()
Q16. Calculate total salary department-wise.
df.groupby('Department')['Salary'].sum()
Q17. Find the maximum salary in each city.
df.groupby('City')['Salary'].max()
Q18. Calculate average, maximum and minimum salary department-wise.
df.groupby('Department')['Salary'].agg(['mean', 'max', 'min'])
Q19. Count employees in each city.
df.groupby('City')['Name'].count()
count() excludes missing Name values. Use .size() to count all rows in each group.
Q20. Find the city with the highest average salary.
df.groupby('City')['Salary'].mean().idxmax()

idxmax() → Returns the label where the maximum value occurs
Q21. Extract the first three characters of a name.
df['Name'].str[:3]
Q22. Check whether a name starts with A.
df[df['Name'].str.startswith('A', na=False)]
Q23. Check whether a name contains Rahul (case-insensitive).

df[df['Name'].str.contains('Rahul', case=False, na=False)]

Q24. Replace spaces with underscores.

df['Name'] = df['Name'].str.replace(' ', '_', regex=False)

Q25. Convert a column into datetime.

df['Date'] = pd.to_datetime(df['Date'])

Q26. Extract the year from a date.

df['Year'] = df['Date'].dt.year

Q27. Calculate employee experience from joining date.

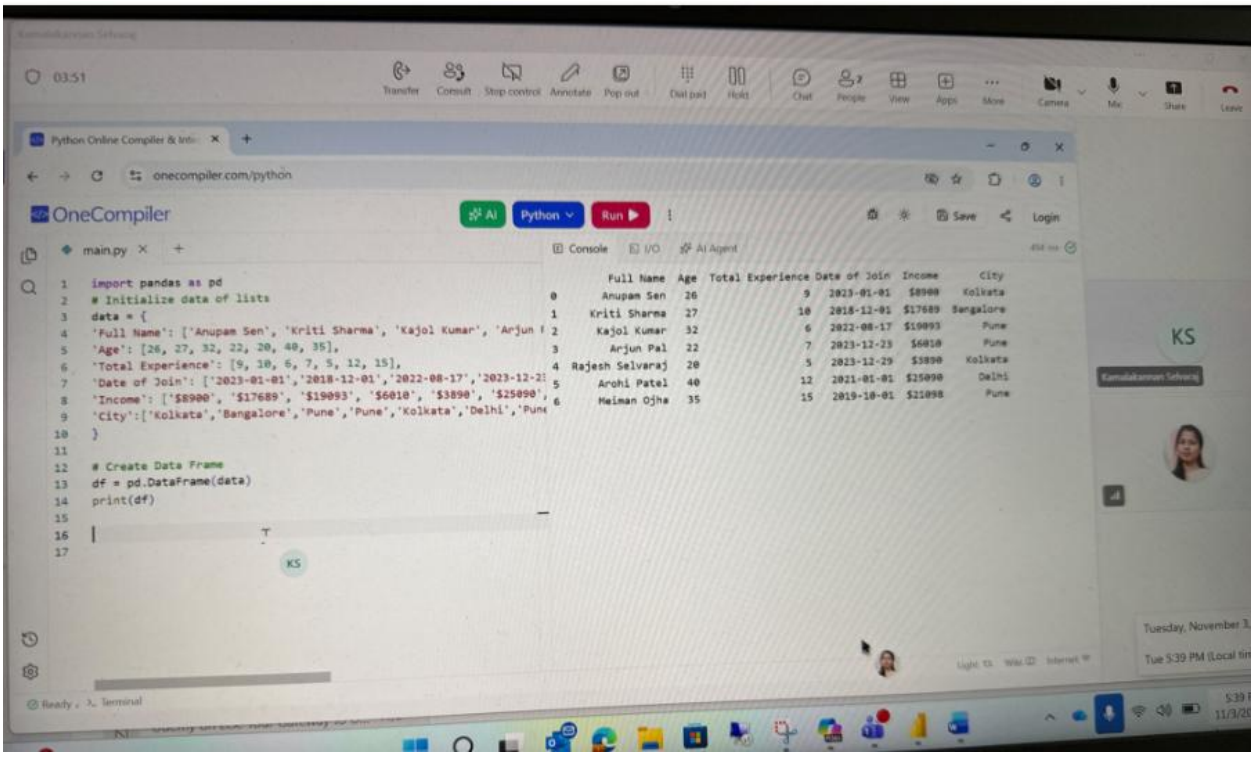
df['Date of Join'] = pd.to_datetime(df['Date of Join'])

df['Experience'] = (pd.Timestamp.today() - df['Date of Join']).dt.days / 365.25

This gives approximate experience in years. Handle missing or future dates if present.

Q28. Calculate the percentage of missing values.

df.isnull().mean() * 100



Q. Extract 2nd row 3rd column

A. df.iloc[1, 2]

Q. If I want to fetch the details of the person who is having a maximum experience, can you print the details of the persons with maximum experience?

max_exp = df['Total Experience'].max()

print(df[df['Total Experience'] == max_exp])

This returns the complete details of all persons having maximum experience.

Q. Print average income of the city Pune

avg_income = df[df['City'] == 'Pune']['Income'].mean()

print(avg_income)

- here in this case income is stored as string .So, first convert it into numeric before calculating the average.

df['Income'] = df['Income'].replace(r'[$,]', '', regex=True)

print(df[df['City'] == 'Pune']['Income'].mean())

You can remove all these special characters using:

```python
df['Income'] = df['Income'].replace(r'[$,₹€@#]', '', regex=True)
```

Q. calculate the current work experience of each employee based on their Date of Joining (DOJ) and today's date.

```
df['Date of Join'] = pd.to_datetime(df['Date of Join'])
today = pd.Timestamp.today()
df['Current Experience'] = (
   (today - df['Date of Join']).dt.days / 365.25).round(2)
print(df)
```

1. Linear Regression – How do you find the best-fit model?

Question: Suppose I have multiple independent variables (X) and one target variable (Y). How do I find the best-fit regression model?

Simple explanation:

Let's say I want to predict an employee's salary based on:

- Years of experience
- Age
- Education
- Previous salary

Here, all four are independent variables (X), and Salary is the target variable (Y).

The best-fit model is the one that makes the smallest prediction errors.

How do I find it?

1. Split data into training and testing sets.
2. Train the model using Linear Regression.
3. Use the OLS (Ordinary Least Squares) method to minimize the sum of squared errors.
4. Evaluate using R^2, Adjusted R^2, MAE and RMSE.
5. Check whether the model performs well on unseen data.

When I use Linear Regression from Scikit-learn, the OLS method is automatically applied during the fit() function to find the best-fit coefficients by minimizing squared errors. I don't need to calculate OLS manually. After fitting the model, I evaluate its performance using R^2, Adjusted R^2, MAE and RMSE on test data to determine how well it generalizes."

Remember: OLS finds the best-fit coefficients; evaluation metrics tell us how good the fitted model is.

1. How do we get the best-fit line?

You can directly use:

```python
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)
```

When you execute `model.fit()`, the model automatically:

- Applies the OLS method.
- Finds the best coefficients (β values).
- Calculates the intercept.
- Fits the regression model by minimizing squared errors.

You can view the coefficients using:

```python
print(model.coef_)
print(model.intercept_)
```

3. Then how do we identify the best model?

For example, suppose you train three models:

Model	R^2	RMSE
Model 1	0.75	5000
Model 2	0.88	3000
Model 3	0.92	2000

Model 3 looks better because it has a higher R^2 and lower RMSE.

However, you should compare these metrics on validation or test data, not just training data.

Q. When we use fit, fit_transform & transform?

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Easy interview answer

"`fit()` is used to train the machine learning model, whereas `fit_transform()` is generally used during preprocessing to learn parameters from the training data and transform it. For Linear Regression, we use `model.fit(X_train, y_train)` to train the model."

Remember:

- `fit()` → Learn.
- `transform()` → Apply transformation.
- `fit_transform()` → Learn and apply transformation together.

Why don't we use fit_transform() on testing data?

In real-world scenarios, when new data comes, we don't know its distribution in advance.
We should use the same preprocessing rules learned from training data.
We use transform() on testing data so that both training & testing datasets use the same scaling parameters. The model should learn only from training data, not testing data
This causes data leakage and makes the training and testing transformations inconsistent.

transform() –Uses the parameters already learned by fit().
transforms new data without learning again.

Data Leakage happens when information from the testing data accidentally gets used during model training.

Suppose you want to predict employee salaries.

You split your data into:

- Training data: 80%
- Testing data: 20%

Now, you apply StandardScaler.

Incorrect approach:

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

Here, the scaler learns parameters separately from both training and testing data.

Correct approach:

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Now, the scaler learns only from training data and applies the same parameters to testing data.

2. How do you reduce the number of independent variables?

Question: I have 20 independent variables. How can I reduce them to 10 important variables to improve model performance?

This process is called Feature Selection.

Think of it this way: Not every column in your dataset is useful for predicting the target variable.

For example, if you are predicting salary, employee ID might not be useful.

Different techniques:

Technique	Simple explanation
Correlation	Remove redundant or highly correlated variables.
VIF	Identify multicollinearity between independent variables.
RFE	Automatically select the most useful features.
Lasso (L1)	Makes some coefficients exactly zero, removing those features.
Ridge (L2)	Reduces coefficient values to control overfitting.
PCA	Combines existing features into fewer new features.

Interview answer:

"I use feature selection techniques such as correlation analysis, VIF, RFE and Lasso regression to identify and remove unnecessary variables. I then train the model again and compare its performance using cross-validation and evaluation metrics."

RFE:RFE (Recursive Feature Elimination)

It is used to select the most important independent variables from a dataset.
It works by:

1. Training a model using all features.
2. Identifying the least important feature.
3. Removing that feature.
4. Repeating the process until the required number of features remains.

Example: If your dataset has 20 input features and you want only 10, RFE selects the best 10 features based on the model you provide.

```
# Step 1: Import libraries
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# Step 2: Separate input and target variables
X = df.drop('Salary', axis=1)
y = df['Salary']

# Step 3: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Create the estimator
model = LinearRegression()
```

```
# Step 5: Apply RFE
rfe = RFE(
    estimator=model,              # Uses Linear Regression to identify important features
    n_features_to_select=10       # Selects the best 10 features from the dataset
)

# Step 6: Fit RFE on training data
rfe.fit(X_train, y_train)          # Evaluates features using training data and selects the best 10

# Step 7: Get selected features
selected_features = X_train.columns[rfe.support_]  # Returns column names of selected features (True =
selected)

print("Selected Features:")
print(selected_features)           # Prints the names of the selected features

# Step 8: Transform training and testing data
X_train_rfe = rfe.transform(X_train)  # Keeps only the 10 selected features in training data
X_test_rfe = rfe.transform(X_test)    # Keeps the same 10 selected features in testing data
```

```python
# Step 9: Train the final model using selected features
final_model = LinearRegression()    # Creates a new Linear Regression model
final_model.fit(X_train_rfe, y_train) # Trains the model using selected features and actual target values

# Step 10: Make predictions
y_pred = final_model.predict(X_test_rfe) # Predicts target values using the test data

# Step 11: Evaluate the model
print("R2 Score:", r2_score(y_test, y_pred))  # Compares actual and predicted values; closer to 1 is
generally better

print("RMSE:", mean_squared_error(y_test, y_pred) ** 0.5)  # Calculates prediction error; lower RMSE is
generally better
```

Complete Machine Learning Workflow: From Model Training to Deployment

Let's understand the entire process with one practical example: Predicting whether a customer will purchase a product (Yes/No).

This is a classification problem because the target variable contains two categories: Yes and No.

1. Understand the problem

Suppose we have customer data:

Age	Salary	Website Visits	Purchased
25	30,000	5	No
35	60,000	12	Yes
45	80,000	15	Yes
22	25,000	3	No
40	70,000	10	Yes

Our objective is to predict whether a new customer will purchase a product.

- **Features (X):** Age, Salary, Website Visits.
- **Target (y):** Purchased.

```python
import pandas as pd

from sklearn.model_selection import (
    train_test_split,
    cross_validate,
    GridSearchCV
)

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (accuracy_score,precision_score,recall_score,f1_score,classification_report,
    confusion_matrix)

import joblib
```

```python
X = df.drop('Purchased', axis=1)
y = df['Purchased']
```

- X contains independent variables (input features).
- y contains the dependent variable (what we want to predict).

3. Split the dataset into training and testing data

Python

```python
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
```

Create different classification models

We will compare three models:

1. Logistic Regression
2. Decision Tree
3. Random Forest

A Pipeline combines preprocessing steps and a machine learning model into one workflow.

Why not use Pipeline for Decision Tree and Random Forest?

Tree-based models generally don't require feature scaling, so we can directly use:

```
models = {
    'Logistic Regression': Pipeline([
        ('scaler', StandardScaler()),
        ('model', LogisticRegression(max_iter=1000))
    ]),

    'Decision Tree': DecisionTreeClassifier(random_state=42),

    'Random Forest': RandomForestClassifier(random_state=42)
}
```

Why use three models?

Different algorithms learn patterns differently. We compare them to identify which performs best on our data.

We use a Pipeline for Logistic Regression so scaling is fitted separately inside each cross-validation fold. Tree-based models generally do not require feature scaling.

5. Perform cross-validation using multiple metrics

Now we evaluate all three models using 5-fold cross-validation.

```
scoring = ['accuracy', 'precision', 'recall', 'f1']

cv_results = {}

for name, model in models.items():

    scores = cross_validate(model,X_train,y_train,cv=5,scoring=scoring)

    cv_results[name] = {
        'Accuracy': scores['test_accuracy'].mean(),
        'Precision': scores['test_precision'].mean(),
        'Recall': scores['test_recall'].mean(),
        'F1 Score': scores['test_f1'].mean()
    }

results_df = pd.DataFrame(cv_results).T

print(results_df)
```

What happens inside 5-fold cross-validation?

Suppose we have 800 training records.

- Divide them into 5 folds.
- Each fold contains 160 records.
- In every iteration, 640 records are used for training and 160 for validation.
- Repeat this process 5 times.

```
cv_results[name] = {
    'Accuracy': scores['test_accuracy'].mean(),
    'Precision': scores['test_precision'].mean(),
    'Recall': scores['test_recall'].mean(),
    'F1 Score': scores['test_f1'].mean()
}
```

et's understand this:

- `cv_results[name]` → Stores results under the current model's name.
- `scores['test_accuracy']` → Gets accuracy scores from all five validation folds.
- `.mean()` → Calculates their average.

he same calculation is performed for precision, recall and F1-score.

6. Compare the cross-validation results and select the best model

Suppose the output looks like this (illustrative scores):

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.84	0.82	0.80	0.81
Decision Tree	0.86	0.85	0.83	0.84
Random Forest	0.91	0.89	0.90	0.89

Here, Random Forest performs best across these metrics.

How do we select the model?

We should not blindly select the model with the highest accuracy.

For example:

- If identifying as many actual buyers as possible is important, focus on Recall.
- If avoiding false purchase predictions is important, focus on Precision.
- If we want a balance between Precision and Recall, focus on F1-score.

For this example, let's select **Random Forest based on its F1-score**.

7. Perform hyperparameter tuning

After selecting the model, we can try to improve its performance by finding better hyperparameters.

Hyperparameters are settings that we provide before ↓ odel training.

7. What happens next?

After comparing the three models, suppose Random Forest performs best.

Python

```
best_model = models['Random Forest']

best_model.fit(X_train, y_train)

y_pred = best_model.predict(X_test)
```

Run

Ask ChatGPT

- `best_model` → Selects Random Forest.
- `.fit()` → Trains it on the complete training dataset.
- `.predict()` → Predicts outcomes for the unseen test dataset.

Remember the overall process:

Either we can directly train the best model as above(step7) or perform hyperparameter as below

Hyperparameters are settings that we provide before model training.

For Random Forest, examples include:

- `n_estimators` : Number of decision trees.
- `max_depth` : Maximum depth of each tree.
- `min_samples_split` : Minimum samples required to split a node.

We can use `GridSearchCV` to test different combinations.

Python Run

```python
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best CV F1 Score:", grid_search.best_score_)
```

Explanation:

- GridSearchCV tries different combinations of hyperparameters.
- It evaluates each combination using 5-fold cross-validation.
- scoring='f1' means we select the combination with the best average F1-score.
- best_params_ gives the best hyperparameter combination.
- best_score_ gives its average cross-validation score.

8. Evaluate the final model on test data

Now we evaluate the tuned model using the test data that we kept separate from cross-validation and hyperparameter tuning.

```python
best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

We can also get a detailed classification report:

```python
print(classification_report(y_test, y_pred))
```

9. Save the final model

Once we are satisfied with its test performance, we can save the model.

```python
import joblib

joblib.dump(best_model, 'customer_purchase_model.pkl')
```

To load it later:

```python
loaded_model = joblib.load('customer_purchase_model.pkl')
```

10. Predict for a new customer

Suppose a new customer has the following details:

- Age: 30
- Salary: 50,000
- Website Visits: 8

Python Run

```
new_customer = pd.DataFrame({
    'Age': [30],
    'Salary': [50000],
    'Website Visits': [8]
})

prediction = loaded_model.predict(new_customer)

if prediction[0] == 1:
    print("Customer will purchase")
else:
    print("Customer will not purchase")
```

The model predicts whether this new customer is likely to purchase the product.

Note: This example assumes the target is encoded as 0 = No and 1 = Yes, and that the input data has already been cleaned and prepared. If your target contains text labels, encode them before using the standard precision, recall and F1 scorers shown above.

---

3. How do you solve the overfitting problem?

Question: Suppose my model performs very well on training data but poorly on testing data. How do I solve this?

First, understand overfitting.

Overfitting example

Training accuracy:**98%**

Testing accuracy:**65%**

The model has learned the training data very well but struggles with new, unseen data.

How can we solve it?

1. How do you solve overfitting in regression?

"To solve overfitting in regression, I use techniques such as:

1. Regularization: Apply Ridge (L2) or Lasso (L1) to control large coefficients.
2. Feature selection: Remove irrelevant features using RFE, correlation or Lasso.
3. Reduce model complexity: Limit tree depth in Decision Tree Regression.
4. Cross-validation: Use K-Fold CV to check model performance across different data splits.
5. More training data: Add more representative data to improve generalization."

2. How do you solve overfitting in classification?

"To solve overfitting in classification, I use:

1. Regularization: Apply L1 or L2 regularization in Logistic Regression.
   For Logistic Regression, L1 and L2 are selected through the penalty parameter.
2. Feature selection: Remove irrelevant features using RFE or correlation.

3. Reduce model complexity: Limit tree depth and prune Decision Trees.
4. Cross-validation: Use Stratified K-Fold to evaluate model performance.
5. More training data: Add more representative training samples."

What is regularization?

Regularization is a technique used to prevent overfitting by adding a penalty to the model's coefficients. It helps control model complexity.

Q4. What are the types of regularization?

There are three commonly discussed types:

1. L1 Regularization – Lasso
2. L2 Regularization – Ridge
3. Elastic Net – Combination of L1 and L2.

Q7. Can we apply Lasso and Ridge in Logistic Regression?

Yes. Logistic Regression supports both L1 and L2 regularization using its `penalty` parameter.

Python

```python
# L1
LogisticRegression(penalty='l1', solver='liblinear')

# L2
LogisticRegression(penalty='l2')
```

Q11. How do you implement Ridge Regression?

Python

```python
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Q12. How do you implement Lasso Regression?

Python

```python
from sklearn.linear_model import Lasso

model = Lasso(alpha=1.0)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Q13. How do you compare Linear, Ridge and Lasso Regression?

```python
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'Lasso': Lasso(alpha=1.0)
}

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    print(name)
    print("R2:", r2_score(y_test, y_pred))
    print("RMSE:", mean_squared_error(y_test, y_pred)**0.5)
```

Q16. How do you identify which features Lasso has removed?

```python
lasso = Lasso(alpha=1.0)
lasso.fit(X_train_scaled, y_train)

print(lasso.coef_)
```

Features having zero coefficients have been eliminated from the model.

Q21. Your Linear Regression model is overfitting. What will you do?

"I will try Ridge and Lasso regularization, perform feature selection, check multicollinearity and use cross-validation to identify the model that generalizes better."

Q22. You have 100 features, but many are irrelevant. Which regularization would you prefer?

"I would try Lasso because it can shrink irrelevant feature coefficients to zero and perform feature selection."

Q23. You have highly correlated independent variables. Which model would you try?

"I would try Ridge Regression because L2 regularization handles multicollinearity by shrinking coefficients. I would also check VIF to identify highly correlated features."

Q24. Your Ridge model performs better than Lasso. What could be the reason?

"Many features may contribute useful information to the target variable. Ridge retains all features while shrinking their coefficients, whereas Lasso may eliminate some features."

8. Find the top five highest-paid employees using Pandas

Scenario: Identify the five employees with the highest salaries.

Python

```python
import pandas as pd

data = {
    'Name': ['A', 'B', 'C', 'D', 'E', 'F'],
    'Salary': [50000, 70000, 45000, 90000, 60000, 85000]
}

df = pd.DataFrame(data)
```

```python
top_5 = df.sort_values(
    by='Salary',
    ascending=False
).head(5)
```

7. Create a boxplot to identify outliers

Scenario: Identify unusual salary values.

Python Run

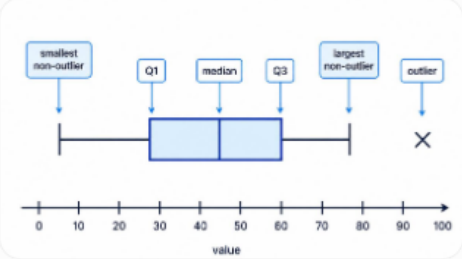
```python
import matplotlib.pyplot as plt

salary = [30000, 35000, 40000, 45000, 50000,
          55000, 60000, 65000, 70000, 200000]

plt.boxplot(salary)

plt.ylabel('Salary')
plt.title('Salary Boxplot')

plt.show()
```



Explanation:

- The box represents the middle 50% of values (IQR).
- The line inside the box represents the median.
- Points beyond the whiskers are potential outliers.
- Investigate outliers before removing them.

Interview answer: I use a boxplot to identify the me quartiles, spread, and potential outliers in

6. Create a histogram to understand data distribution

Scenario: Understand the distribution of employee salaries.

Python

Run

```python
import matplotlib.pyplot as plt

salary = [30000, 35000, 40000, 45000, 50000,
          55000, 60000, 65000, 70000, 75000]

plt.hist(salary, bins=5, edgecolor='black')

plt.xlabel('Salary')
plt.ylabel('Frequency')
plt.title('Salary Distribution')

plt.show()
```



Explanation:

- Histogram shows the frequency distribution of numerical data.
- X-axis represents value ranges (bins).

4. Perform a Chi-square test

Scenario: Check whether gender is associated with product preference.

Python

```python
from scipy.stats import chi2_contingency

# Rows: Male, Female
# Columns: Product A, Product B
observed = [
    [30, 20],
    [15, 35]
]

chi2, p_value, dof, expected = chi2_contingency(observed)

print("Chi-square:", chi2)
print("P-value:", p_value)
print("Degrees of freedom:", dof)
print("Expected frequencies:")
print(expected)

if p_value < 0.05:
    print("There is an association")
else:
    print("No significant association")
```

Explanation:

- Chi-square tests the association between categorical variables.
- It compares observed frequencies with expected frequencies.
- It does not establish causation.

3. Perform ANOVA using SciPy

Scenario: Compare average sales across three regions.

Python

```python
from scipy.stats import f_oneway

north = [20, 25, 30, 28, 32]
south = [35, 38, 40, 42, 39]
east = [15, 18, 20, 22, 25]

f_stat, p_value = f_oneway(north, south, east)

print("F-statistic:", f_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Significant difference between groups")
else:
    print("No significant difference")
```

Explanation:

- ANOVA compares means across three or more groups.
- A significant result means at least one group mean differs.
- A post-hoc test, such as Tukey HSD, helps identify which groups differ.

1. Calculate correlation between two columns

Scenario: Find the correlation between advertising spend and sales.

```python
import pandas as pd

data = {
    'Advertising': [10, 20, 30, 40, 50],
    'Sales': [15, 25, 35, 45, 55]
}

df = pd.DataFrame(data)

correlation = df['Advertising'].corr(df['Sales'])

print(correlation)
```

Output: `1.0`

Explanation:

- +1 = Perfect positive correlation.
- -1 = Perfect negative correlation.
- 0 = No linear correlation.

LOC & ILOC

Both loc and iloc are used for selecting rows and columns in Pandas. loc is label-based, whereas iloc is integer-position-based. For example, I use loc to filter employees whose age is greater than 30, and iloc to select the second row or third column based on position."

2. Select a specific column

```python
df.loc[:, 'Name']
```

Output: All employee names.

3. Select multiple rows

```python
df.loc[0:2]
```

Output: Rows 0, 1 and 2.

4. Filter based on a condition

```python
df.loc[df['Age'] > 30]
```

4. Select the first three rows

Python

```
df.iloc[0:3]
```

Output: Rows at positions 0, 1 and 2.

5. Select the first three rows and first two columns

Python

```
df.iloc[0:3, 0:2]
```

```
print( 'i is not present' )
```

[ ]:
```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

Count the number of digits of a number -> 1234 -> output 4

4]:
```
#conver to string and use len
n = int(input())
print(len(str(n)))
```

```
86586
5
```

5]:
```
n = int(input())
count = 0
while n!=0:
    n = n//10
    count += 1
print(count)

# 4567 // 10 = 456 → count = 1
# 456 // 10 = 45 → count = 2
# 45 // 10 = 4 → count = 3
# 4 // 10 = 0 → count = 4
```

```
4567
4
```

Deep Learning: It works with unstructured data like text, image, audio, video.

What does ML model do?

A machine learning model learns patterns and relationships from historical data during training. Once it has learned those patterns, it can use them to make predictions or decisions on new data that it has never seen before."

- **what is difference between NumPy & pandas & how have you used it?**
  NumPy is mainly used for numerical computations and works with arrays, while Pandas is used for data manipulation and works with structured data like tables (Data Frames).
  NumPy is faster for mathematical operations, whereas Pandas provides flexibility for data cleaning, filtering, and analysis."
  "In my projects, I used:
  Pandas for data cleaning, handling missing values, filtering data, and EDA"

```python
import numpy as np

df['age'] = np.where(df['age'] < 0, np.nan, df['age'])
```

✅ What this code does

> "This code replaces invalid negative values in the `age` column with missing values (`NaN`)."

Pandas is used for handling missing values, while NumPy is used for conditional operations and replacements."

🔹 4. Feature Engineering → Both

```python
df['new_col'] = df['salary'] * 12
```

Ask ChatGPT

👉 NumPy:

```python
df['risk'] = np.where(df['balance'] > 50000, 1, 0)
```

👉

> "Pandas helps create new columns, and NumPy helps apply fast conditional logic."

- **If you are importing pandas then is it necessary to import NumPy as well?**

No, it's not mandatory to import NumPy separately because Pandas is built on top of NumPy.
However, I usually import NumPy because it provides additional mathematical functions and better control over arrays.

- How have you used scikit learn in project(eg micro credit defaulter project)?
  I used Scikit-learn for the complete machine learning pipeline:

1. Data splitting using train_test_split
2. Model building using algorithms like Logistic Regression, Decision Tree, and Random Forest
3. Model evaluation using accuracy score, confusion matrix, and ROC-AUC
4. Feature selection and preprocessing where required"
   a. 👉 If they ask specifically about credit defaulter:
   b. "In the credit defaulter project, I used classification models to predict whether a customer will default or not based on financial and demographic features.

In my project, I used Scikit-learn for the complete machine learning workflow. I split the data using train_test_split, trained models like Logistic Regression and Random Forest, and evaluated them using accuracy, confusion matrix, and ROC-AUC. This helped me select the best-performing model

- How you decided whether the customer will churn or not?
- How you decide ,which model is best among 5 model using roc-auc curve?

1. **What is Python, and why is it used in data analytics?** Python is an open source high-level, interpreted programming language with dynamic semantics known for its simplicity and readability. It is widely used in data analytics due to its extensive libraries and frameworks, such as Pandas, NumPy, Matplotlib, and SciPy, which facilitate data manipulation, analysis, and visualization. Python's versatility allows for easy integration with other technologies, making it a preferred choice for data analysts.,

- **Key Features:**
  - **Interpreted:** Python code is executed line by line, which makes it easier to debug and test.

- **Object-Oriented:** Python is designed around the concept of "objects," which are data structures that can contain both data and functions.
- **Dynamic Typing:** You don't need to declare the type of a variable (e.g., integer, string) before using it.
- **Open Source:** Python is free to use and distribute, and anyone can contribute to its development.

What are the key libraries in Python data analyst?

- **Pandas**: Pandas is particularly useful for data manipulation, cleaning, and exploration using DataFrames, which are similar to tables in databases or Excel spreadsheets.

- Pandas is well-suited for working with tabular data, such as spreadsheets or SQL tables.
- Reading and writing data from various file formats like CSV, Excel, and SQL databases.

- **NumPy**: Used for numerical computing in Python. It provides support for large multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.
- **Matplotlib**: Used for creating static, interactive, and animated visualizations in Python.
- **Seaborn**: Built on top of Matplotlib, it provides a high-level interface for drawing attractive and informative statistical graphics.
- **SciPy**: **SciPy** is a powerful Python library that builds on top of **NumPy** and provides additional functionality for numerical operations. Used for scientific and technical computing, including optimization, integration, interpolation, eigenvalue problems, and other linear algebra tasks.
- **Scikit-learn**: A machine learning library for Python that supports various supervised and unsupervised learning algorithms.

2. Can you explain the difference between NumPy and Pandas? NumPy (Numerical Python) and Pandas are two essential Python libraries for data analysis.

- **NumPy:** Primarily used for numerical computations, NumPy provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions. It is best suited for performing element-wise operations and linear algebra.
- **Pandas:** Built on top of NumPy, Pandas offers more flexible data structures like DataFrames and Series. It is used for data manipulation and analysis, allowing users to handle data with missing values, perform group operations, and more. Pandas is particularly useful for working with labeled data and data frames.

3. What is a Python dictionary, and how is it different from a list?
A Python dictionary is an unordered collection of key-value pairs, where each key is unique and used to access the corresponding value, value can be duplicated. It is defined using curly braces {} and is mutable, meaning its contents can change over time.
In contrast, a list is an ordered collection of elements defined using square brackets []. Lists allow duplicate elements and are also mutable. The main difference between the two is that dictionaries use keys for indexing, while lists use numerical indices.

Adding Elements into List

We can add elements to a list using the following methods:

- **append():**- Adds an element at the end of the list.
- **extend():** Adds multiple elements to the end of the list.
- **insert():** Adds an element at a specific position.

```
# Initialize an empty list
a = []

# Adding 10 to end of list
a.append(10)
print("After append(10):", a)

# Inserting 5 at index 0
a.insert(0, 5)
print("After insert(0, 5):", a)

# Adding multiple elements  [15, 20, 25] at the end
a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)
```

```
After append(10): [10]
After insert(0, 5): [5, 10]
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
```

Removing Elements from List

We can remove elements from a list using:

- **remove():** Removes the first occurrence of an element.
- **pop():** Removes the element at a specific index or the last element if no index is specified.
- **del statement:** Deletes an element at a specified index.

```
a = [10, 20, 30, 40, 30,50]

# Removes the first occurrence of 30
a.remove(30)
print("After remove(30):", a)

# Removes the element at index 1 (20)
popped_val = a.pop(1)
print("Popped element:", popped_val)
print("After pop(1):", a)

# Deletes the first element (10)
del a[0]
print("After del a[0]:", a)
```

4. How would you handle missing data in a dataset using Python?

Handling missing data is crucial in data analysis. In Python, this can be done using the Pandas library. Here are a few common methods:

Check for missing values

- **df.isnull()** Returns a DataFrame of True/False indicating missing values
- **df.isnull().sum())** Returns the count of missing values in each column

Removing Missing Values: Use dropna() to remove rows or columns with missing values.

df.dropna() # Drops rows with any missing values

df.dropna(axis=1) # Drops columns with any missing values

Filling/ Imputing Missing Values: Use **fillna()** to replace missing values with a specified value, such as the mean, median, or mode.

Fill with a specific value (e.g., 0)

df.fillna(0) # Fill all missing values with 0

Fill with the mean, median, or mode:

The median is the middle value of a data set when it is ordered from smallest to largest. If there is an odd number of values, the median is the middle value. If there is an even number of values, the median is the average of the two middle values.

The mode is the value that appears most frequently in a data set.

```
numerical_columns = ['No of Bathrooms', 'Flat Area (in Sqft)','Lot Area (in Sqft)',
                     'Area of the House from Basement (in Sqft)','Latitude',
                     'Longitude','Living Area after Renovation (in Sqft)']
```

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values = np.nan, strategy = 'median')
Raw_Housing_Data[numerical_columns] = imputer.fit_transform(Raw_Housing_Data[numerical_columns])
```

df['Age'] = df['Age'].fillna(df['Age'].mean()) ***#for specific column***

df.fillna(df.mean()) Fill missing values with the column mean

df.fillna(df.median()) Fill missing values with the column median

df.fillna(df.mode().iloc[0]) # mode() returns a DataFrame, so we use .iloc[0] Fill missing values with the column mode (for **categorical data)**

```
imputer = SimpleImputer(missing_values = np.nan, strategy = 'most_frequent')
Raw_Housing_Data['Zipcode'] = imputer.fit_transform(Raw_Housing_Data['Zipcode'])
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-8-c6b9f70ed569> in <module>
      1 imputer = SimpleImputer(missing_values = np.nan, strategy = 'most_frequent')
----> 2 Raw_Housing_Data['Zipcode'] = imputer.fit_transform(Raw_Housing_Data['Zipcode'])

~\anaconda3\lib\site-packages\sklearn\utils\validation.py in check_array(array, accept_sparse, accept
_large_sparse, dtype, order, copy, force_all_finite, ensure_2d, allow_nd, ensure_min_samples, ensure_
min_features, estimator)
    622                     "Reshape your data either using array.reshape(-1, 1) if "
    623                     "your data has a single feature or array.reshape(1, -1) "
--> 624                     "if it contains a single sample.".format(array))
    625
    626         # in the future np.flexible dtypes will be handled like object dtypes

ValueError: Expected 2D array, got 1D array instead:
array=[98178. 98125. 98028. ... 98144. 98027. 98144.].
Reshape your data either using array.reshape(-1, 1) if your data has a single feature or array.reshap
e(1, -1) if it contains a single sample.
```

While imputing for categorical column, if we got this error

```
: Raw_Housing_Data['Zipcode'].shape
: (21609,)
```

Here clearly, we can see that, column that we are passing above is single dimensional array. So, we can reshape this to avoid the error as below

```
column = Raw_Housing_Data["Zipcode"].values.reshape(-1,1)
column.shape
```

It signifies that the reshape function will adjust the number of rows by itself.

Forward-fill (propagate the last valid value forward):
df_ffill = df.fillna(method='ffill')
Backward-fill (use the next valid value to fill the missing one):
df_bfill = df.fillna(method='bfill')

Imputation: Imputing with Scikit-learn:

For more sophisticated imputation techniques (especially when you want to use machine learning methods), Scikit-learn provides an **Imputer** class.

```
from sklearn.impute import SimpleImputer

# Create an imputer object to replace missing values with the mean of each column
imputer = SimpleImputer(strategy='mean')  # You can also use 'median', 'most_frequent', etc

# Fit and transform the data
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
print(df_imputed)
```

5. What are Python decorators, and how are they used?
Decorators are a powerful feature in Python that allows the modification of functions or methods without changing their actual code. They are defined using the @decorator_name syntax above a function definition. Decorators are commonly used for logging, enforcing access control, instrumentation, and caching.

6. Can you explain the concept of 'group by' in Pandas?
The groupby method in Pandas is used to group data based on one or more columns. It splits the data into separate groups, applies a function to each group independently, and then combines the results. This method is particularly useful for aggregating data, such as calculating the sum, mean, or count of a grouped dataset. For example, df.groupby('column_name').sum() would sum up the values of each group.

7. What is a lambda function in Python?
A lambda function is a small anonymous function defined using the lambda keyword. It can have any number of input parameters but can only have one expression.
Lambda functions are often used for short-term tasks that do not require a full function definition.
Lambda function to add two numbers

```
add = lambda x, y: x + y
print(add(5, 3))  # Output: 8
```

Using a lambda function with the `map()` function to square each number in a list:

```python
numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(lambda x: x ** 2, numbers))

print(squared_numbers)  # Output: [1, 4, 9, 16, 25]
```

8. How do you optimize a Python code for performance?

Optimizing Python code involves several strategies:

Using Efficient Data Structures: Choosing the right data structures (e.g., sets for membership testing, dictionaries for lookups) can significantly speed up code.

Avoiding Global Variables: Minimize the use of global variables as they can slow down the program.

List Comprehensions: Use list comprehensions instead of traditional loops for creating lists, as they are faster and more readable.

Profiling: Use profiling tools like cProfile to identify bottlenecks in the code.

Using Built-in Functions: Leverage Python's built-in functions and libraries, as they are optimized for performance.

Q. What is local & global variable in python?

global variables are declared outside any function and accessible throughout the program, while local variables are defined within a function and only accessible within that function.

Example of Local Variable:

```python
def my_function():
    x = 10  # Local variable
    print(x)  # Accessing the local variable

my_function()  # Output: 10

# Trying to access x outside the function will result in an error:
# print(x)  # NameError: name 'x' is not defined
```

Example of Global Variable:

```
python

x = 10  # Global variable

def my_function():
    print(x)  # Accessing the global variable

my_function()  # Output: 10
print(x)  # Output: 10, global variable can be accessed here too
```

Tuples in Python

```
A tuple is similar to a Python list in terms of indexing, nested objects, and repetition but the main difference between both is
Python tuple is immutable, unlike the Python list which is mutable.
```

- Like Lists, tuples are ordered and we can access their elements using their index values
- We cannot update items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.

]:
```
# Note : In case of list, we use square brackets [].
#  Here we use round brackets ()
t = (10, 20, 30)

print(t)
print(type(t))
```

]:
```
t = (1, 2, 3, 4, 5)

# tuples are indexed
print(t[1])
print(t[4])

# tuples contain duplicate elements
t = (1, 2, 3, 4, 2, 3)
print(t)
```

```
#traversing Items of Python Tuples
#Like List Traversal, we can traverse through a tuple using for loop.
# Define a tuple
t = (1, 2, 3, 4, 5)

# Traverse through each item in the tuple
for x in t:
    print(x, end=" ")  # end=" ",  ensures that the items are printed on the same line, separated by a space.
```

```
1 2 3 4 5
```

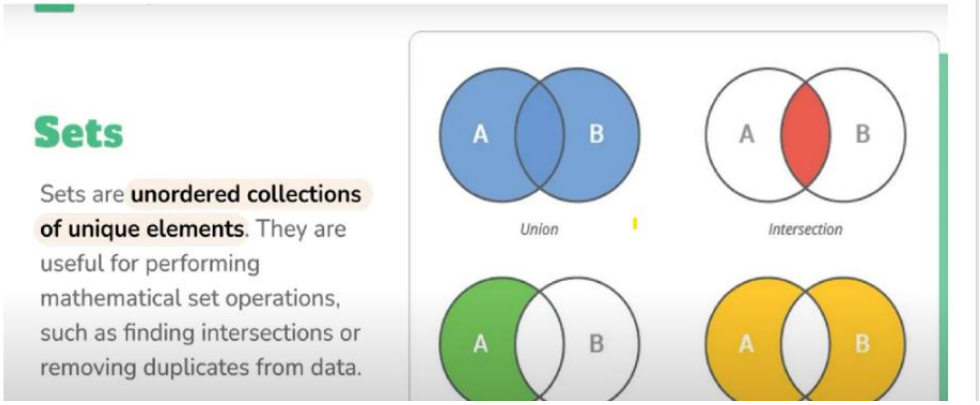
```
#Concatenation of Tuples
# Code for concatenating 2 tuples
t1 = (0, 1, 2, 3)
t2 = ('python', 'geek')

# Concatenating above two
print(t1 + t2)
```

```
(0, 1, 2, 3, 'python', 'geek')
```

Set in Python

set is an unordered collection of multiple items having different datatypes. In Python, sets are mutable, unindexed and do not contain duplicates. The order of elements in a set is not preserved and can change.



Typecasting in Python means converting a variable from one data type to another, such as from `int` to `float`, `str` to `int`, etc.

Typecasting objects into sets in Python refers to converting various data types into a set. Python provides the set() constructor to perform this typecasting, allowing us to convert lists, tuples and strings into sets.

```
# Typecasting list into set
li = [1, 2, 3, 3, 4, 5, 5, 6, 2]
set1 = set(li)
print(set1)

# Typecasting string into set
s = "GeeksforGeeks"
set1 = set(s)
print(set1)

# Typecasting dictionary into set
d = {1: "One", 2: "Two", 3: "Three"}
set1 = set(d)
print(set1)
```

Dictionary Data Type

unlike other Python Data Types that hold only a single value as an element, a Dictionary holds a key: value pair. Key-value is provided in the dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon : , whereas each key is separated by a 'comma'.

values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable. The dictionary can also be created by the built-in function dict().

Note – Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

9. What is the difference between a shallow copy and a deep copy in Python?

Shallow Copy: A shallow copy creates a new object but does not create copies of nested objects. It only copies reference to the original objects, meaning changes to the nested objects affect both copies. It can be created using the copy() method or the copy module.

Deep Copy: A deep copy creates a new object along with new copies of nested objects, ensuring that changes in the copied object do not affect the original. It can be created using the deepcopy() method from the copy module.

10. How would you merge two DataFrames in Pandas?

In Pandas, merging two DataFrames can be done using the merge() function, similar to SQL joins. The function allows you to specify the type of join (inner, outer, left, right) and the key(s) to merge on. For example, pd.merge(df1, df2, on='key') merges df1 and df2 on the column key.

📌 Example: Left Join

```python
pd.merge(df1, df2, on='id', how='left')
```

→ Keeps all rows from `df1`, adds matching `score` from `df2`, `NaN` if no match.

11. What is the use of the map () function in Python?

The map() function in python applies a given function to each item of an iterable (like a list, tuple, or string) and returns a new iterator containing the results, without using an explicit for loop. It is commonly used for applying a function to each element of a list. For example, map(lambda x: x*2, [1, 2, 3, 4]) returns [2, 4, 6, 8].

12. Can you explain the concept of list comprehension in Python?

List comprehension in Python is a concise and efficient way to create new lists by applying an expression to each item in an existing iterable (like a list, tuple, or range), For example, [x**2 for x in range(10)] creates a list of squares of numbers from 0 to 9.

Using list comprehension to get even numbers
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = [x for x in numbers if x % 2 == 0]

Q. Difference between list comprehension and lambda function.

List Comprehension:

Use it when you need to create a new list based on some transformation or filtering of elements from an existing iterable (like a list, tuple, or range).

Lambda Function:

Use it when you need a small, anonymous function for a simple operation that will be used only once or passed to another function (e.g., map(), filter(), or sorted()).

13. How do you handle large datasets in Python?

Handling large datasets in Python can be challenging due to memory constraints. Some strategies include:

Chunking: Loading data in chunks using libraries like Pandas (read_csv with the chunksize parameter) allows you to process large files in smaller pieces.

Read data in chunks: You can process large CSV files in chunks rather than reading everything into memory at once.

```python
chunk_size = 100000  # Adjust based on memory size
for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    process(chunk)
```

Reduce Memory Usage by Optimizing Data Types: Use efficient data types and data structures to minimize memory usage, such as using float32 instead of float64 when possible.

Use `dtype` specification: Specify appropriate data types to reduce memory usage.

```python
df = pd.read_csv('large_file.csv', dtype={'column_name': 'int32'})
```

```python
# Example DataFrame with inefficient datatypes
data = {'A': [1, 2, 3, 4],
        'B': [5.0, 6.0, 7.0, 8.0]}
df = pd.DataFrame(data)
df['A'].dtype,df['B'].dtype ,print("Memory usage Before conversion:",df.memory_usage().sum())
```

```
Memory usage Before conversion: 192
(dtype('int64'), dtype('float64'), None)
```

```python
# Convert to more memory-efficient datatypes
df['A'] = pd.to_numeric(df['A'], downcast='integer')
df['B'] = pd.to_numeric(df['B'], downcast='float')
df['A'].dtype,df['B'].dtype,print("Memory usage after conversion:",df.memory_usage().sum())
```

```
Memory usage after conversion: 148
(dtype('int8'), dtype('float32'), None)
```

```python
# Typecasting
df['A'] = df['A'].astype('int32')
df['B'] = df['B'].astype('float32')
df['A'].dtype,df['B'].dtype,print("Memory usage after conversion:",df.memory_usage().sum())
```

```
Memory usage after conversion: 160
(dtype('int32'), dtype('float32'), None)
```

14. How would you detect and remove duplicate rows in a dataset?

Detect Duplicates: Use df.duplicated() to identify duplicates.
This checks for **duplicate rows across the entire DataFrame**
(i.e., **all columns** must match exactly).
Remove Duplicates: Use df.drop_duplicates().

3. Display all occurrences of duplicate values

```python
df[df['column_name'].duplicated(keep=False)]
```

5. Remove duplicate rows based on a column

```python
df.drop_duplicates(subset=['column_name'], keep='first')
```

15. What are some common Python libraries used in data analysis?

- **Pandas:** For data manipulation and analysis.
- **NumPy:** For numerical computations.
- **Matplotlib:** For data visualization.
- **Seaborn:** For statistical data visualization.
- **SciPy:** For scientific and technical computing.

- **Scikit-learn:** For machine learning and predictive analysis.
- **Statsmodels:** For statistical modeling and hypothesis testing.

16. How do you group data in Pandas and perform aggregations?
Answer: Use groupby() for grouping and aggregation:

```
import pandas as pd

data = {
   'Category': ['A', 'B', 'A', 'B', 'A'],
   'Values': [10, 20, 30, 40, 50]
}
df = pd.DataFrame(data)

# Group by Category and calculate the sum of Values
grouped = df.groupby('Category')['Values'].sum()
print(grouped)
```

17. What is the purpose of the groupby() function in Pandas?
The groupby() function in Pandas is used to split data into groups based on some criteria. It is often followed by an aggregation function like sum(), mean(), count(), etc., to apply a function to each group independently. This is particularly useful for summarizing and analyzing large datasets, such as calculating the total sales for each product category.

18.You have a dataset with 1 million rows. How would you handle it efficiently?

Read Data in Chunks: Use pd.read_csv() with the chunksize parameter.

Use Dask: Dask is a parallel computing library in Python that helps you work with large datasets and speed up computations

```
pip install dask[complete]
```

🧪 Example (Dask DataFrame)

python Copy

```python
import dask.dataframe as dd

# Load a large CSV file using Dask
df = dd.read_csv('your_big_file.csv')

# Just like pandas!
print(df.head())
print(df.describe())

# Group and compute
result = df.groupby('category').amount.sum().compute()
print(result)
```

✅ `compute()` triggers the actual computation (lazy evaluation like Spark)

Optimize Data Types: Convert columns to appropriate data types to reduce memory usage.
Indexing: Use indexing for faster lookups and filtering.

20.What is the difference between 'merge ()', 'join()', and 'concat()' in Pandas?

merge(): merge() is used to combine two DataFrames based on a common column (or index), similar to SQL joins (like inner join, outer join, left join, and right join).

It allows you to specify how you want to merge (inner, outer, left, or right) using the how parameter.

```
# Merge on the 'id' column (similar to SQL JOIN)
result = pd.merge(df1, df2, on='id', how='inner')
print(result)
```

join(): join() is used to combine two DataFrames based on the index or a column. It is more convenient when working with DataFrames that share the same index, or when you want to join on indices directly.

Join on index: The join is done based on the index (default behavior of join()).

```
# Join on index
result = df1.join(df2, how='inner')
print(result)
```

concat(): concat() is used to combine two or more DataFrames along a particular axis (either rows or columns). It doesn't require a common column or index for merging like merge() or join().You can specify the axis with the axis parameter: axis=0 for rows (default) and axis=1 for columns.

ignore_index: If True, it resets the index in the resulting DataFrame.

```
# Concatenate DataFrames vertically (along rows)
result = pd.concat([df1, df2], ignore_index=True)
print(result)
```

22.What is the difference between a Python list and a NumPy array?

Data Type: A list can store elements of different data types, including integers, strings, floats, or even other lists. Array stores elements of the same data type. This uniformity allows NumPy to perform operations efficiently on large datasets.

Operations: NumPy supports vectorized operations, whereas lists require explicit loops for element-wise operations.

Dimensionality: A Python list can be a 1D collection (a simple list), or it can be a nested list (for multi-dimensional data), handled manually. NumPy supports multi-dimensional arrays (1D, 2D, 3D, etc.) natively. It provides powerful tools for handling and manipulating data in various dimensions (like matrices or higher-dimensional arrays).

24.Explain the difference between 'apply()', 'map()', and 'applymap()' in Pandas.

apply(): Used for applying a function along an axis (rows or columns) of a DataFrame.

map(): Used for element-wise operations on a Pandas Series.

applymap(): Used for element-wise operations on a DataFrame.

Feature	apply()	map()	applymap()
Use case	Can be used on Series and DataFrames	Used only on Series	Used only on DataFrames
Axis	Works along a specific axis (axis=0 for columns, axis=1 for rows)	Applies a function element-wise to a Series	Applies a function element-wise to a DataFrame
Type of operation	More flexible, works on rows/columns of a DataFrame	Element-wise operation on a Series (no axis argument)	Element-wise operation on a DataFrame
When to use	Use when you need to perform operations on columns or rows of a DataFrame, or when working with Series	Use when you need to apply a function or map values element-wise to a Series	Use when you need to apply a function element-wise to all values in a DataFrame

Slicing

Slicing in Python refers to extracting a portion of a sequence like a list, string, or tuple by specifying the start and end indexes. The syntax for slicing is string[start:end], where start starting index and end is stopping index (excluded).

Sequence Data Types

Python String: Python treats anything inside quotes as a string. This includes letters, numbers, and symbols. Python has no character data type so single character is a string of length 1.

- Multi-line Strings

```
s = """I am Learning
Python String on GeeksforGeeks"""
```

Strings in Python are immutable. This means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like concatenation, slicing, or formatting to create new strings based on the original.

```
s = "geeksforGeeks"

# Trying to change the first character raises an error
# s[0] = 'G'  # Uncommenting this line will cause a TypeError

# Instead, create a new string
s = "G" + s[1:]
print(s)
```

```
GeeksforGeeks
```

Updating a String

To update a part of a string we need to create a new string since strings are immutable.

```
s = "hello geeks"

# Updating by creating a new string
s1 = "H" + s[1:]

# replacnig "geeks" with "GeeksforGeeks"
s2 = s.replace("geeks", "GeeksforGeeks")
print(s1)
print(s2)

#For s1, The original string s is sliced from index 1 to end of string and then concatenate “H” to create a new string s1.
#For s2, we can created a new string s2 and used replace() method to replace ‘geeks’ with ‘GeeksforGeeks’.
```

Common String Methods

```
#len(): The len() function returns the total number of characters in a string.
s = "GeeksforGeeks"
print(len(s))
```

```
#upper() and lower(): upper() method converts all characters to uppercase. lower() method converts all characters to lowercase.
s = "Hello World"

print(s.upper())   # output: HELLO WORLD

print(s.lower())   # output: hello world
```

```
#strip() :strip() removes leading and trailing whitespace from the string
#replace(): 0r2


#replace(old, new)replaces all occurrences of a specified substring with another.
s = "   Gfg   "

# Removes spaces from both ends
print(s.strip())

s = "Python is fun"

# Replaces 'fun' with 'awesome'
print(s.replace("fun", "awesome"))
```

**********.

In Python, loops are used to execute a block of code repeatedly.

loops in python

for loop & while loop

- A for loop in Python is used to iterate over a sequence (list, tuple, set, dictionary, and string).
- while loop is loop which continue to execute until a condition is satisfied

```
factorial = 1
for i in range(1,11):
    print("value of i=" ,i)
    print("multiplying ", factorial ,"with ", i)
    factorial =factorial*i
    print("factorial",factorial)
```

```
correct_password ="abcd"
input_password = input("Enter your Password :")
while input_password != correct_password:
    print("incorrect password :try again")
    input_password = input("Enter your Password :")
print("successfully logged in")

#until i keep on writing incorrect password,it will keep executing
```

File Handling Operation

four primary functions used for file handling in Python are:

- open() : Opens a file and returns a file object.
- read() : Reads data from a file.
- write() : Writes data to a file.
- close() : Closes the file, releasing its resources.

Reading a File in Read Mode (r)

```
#opening a file
file = open("C:/Users/User/Desktop/python.txt", "r")
content = file.read()  #reading a file
print(content)
file.close() #which closes the file after reading it, which is necessary to free up system resources.
```

```
Hello, shama Hello, zaryab qudrat
```

Writing to a File (w mode) write mode

if we open an existing file in w or w+ mode ,then all of its pre existing content will be deleted Writing to a file is done using file.write() which writes the specified string to the file. If the file exists, its content is erased. If it doesn't exist, a new file is created.

```
file = open("C:/Users/User/Desktop/python.txt", "w")
file.write("Hello Zaryab")
file.close()

#check the file in given path ,hello shama will be written in it
```

Writing to a File (w+ mode) write & read mode

```
file = open("C:/Users/User/Desktop/python.txt", "w+")
f=file.write("Hello, shama")
print(f)  #Returns the number of characters written, not the file object anymore


#If you want to write and then read the content from the same file:
f=file.seek(0)  # Move the file pointer to the start
x= file.read()
x
#file.read()
```

```
12
```

```
'Hello, shama'
```

Writing to a File in Append Mode (a) ¶

It is done using file.write() which adds the specified string to the end of the file without erasing its existing content.

```
file = open("C:/Users/User/Desktop/python.txt", "a")
file.write(" Hello, zaryab qudrat")
file.close()

#checking the above file by opening it in read mode
file = open("C:/Users/User/Desktop/python.txt", "r")
file.read()
```

```
'Hello, shama Hello, zaryab qudrat'
```

**********.

OUTLIERS IN machine learning

Outliers are values in your dataset that are unusually high or low compared to the rest of the data. They can **distort** your model's performance, so it's important to detect and handle them properly.

- **Detecting Outliers Using Z-score (Standard Deviation)**

The **Z-score** measures how far a data point is from the **mean in terms of standard deviations**.

```python
import numpy as np

mean = df['values'].mean()
std = df['values'].std()

# Z-score formula
df['z_score'] = (df['values'] - mean) / std

# Consider values with z-score > 3 or < -3 as outliers
outliers_z = df[np.abs(df['z_score']) > 3]
print("Outliers (Z-score method):\n", outliers_z)
```

Z-score method uses **-3 to +3 as the threshold** for detecting outliers because of how data behaves in a **normal distribution**.

Why ±3 Is Used

In a **normal (Gaussian) distribution**, there is a rule called the **68–95–99.7 rule**.

Range from Mean	Percentage of Data
±1 standard deviation	~68%
±2 standard deviations	~95%
±3 standard deviations	~99.7%

This means:

- **99.7% of the data lies between -3 and +3**
- Only **0.3% of values fall outside this range**

So values beyond this range are **very rare** and are considered **outliers**.

2. IQR Method (Interquartile Range)

IQR uses **quartiles instead of mean.**

Formula:

```
IQR = Q3 - Q1
```

Outlier limits:

```
Lower limit = Q1 - 1.5 × IQR
Upper limit = Q3 + 1.5 × IQR
```

Example

Dataset:

```
10, 12, 13, 14, 15, 100
```

Quartiles:

```
Q1 = 12
Q3 = 15
IQR = 3
```

Upper limit:

```
15 + (1.5 × 3) = 19.5
```

Since **100 > 19.5 → outlier**

Situation	Method
Normally distributed data	Z-score
Skewed data	IQR
Small dataset	IQR
Extreme values present	IQR

Z-score and IQR are both methods to detect outliers. Z-score measures how many standard deviations a value is from the mean and works well for normally distributed data. IQR uses quartiles and is more robust for skewed distributions.

✅ 3. Visualizing Outliers

📌 Boxplot (helps visualize IQR-based outliers)

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df['values'])
plt.show()
```

**********.

understand the basic statistics of your dataset

In **machine learning**, `df.describe()` is a very useful tool during the **exploratory data analysis (EDA)** phase. It helps you quickly understand the distribution and spread of your numeric data,which is crucial for identifying **outliers**, **skewness**, and how the **mean** and **median** compare

It returns a summary of the numeric columns in your DataFrame:

Metric	Meaning
`count`	Number of non-null values
`mean`	Average of the values
`std`	Standard deviation (spread of data)
`min`	Minimum value
`25%`	1st quartile (Q1) — 25% of data below this
`50%` (median)	2nd quartile (Q2) — middle value of the data
`75%`	3rd quartile (Q3) — 75% of data below this
`max`	Maximum value

- Outliers are values that are far from the rest of the data.
- Check for **large gaps** between `75%` and `max`, or `min` and `25%`.

Skew Type	Mean vs Median	Outlier Direction	Outliers Present?
Positive Skew	Mean > Median	High/right side	Likely (e.g., large values)
Negative Skew	Mean < Median	Low/left side	Likely (e.g., very small values)
No Skew	Mean ≈ Median	Symmetric	Possibly none or balanced

ERROR METRIX IN MACHINE LEARNING

In machine learning, **error metrics** help us measure how well a model performs — they tell us **how far off** our predictions are from the actual values.

🔢 Types of Error Metrics

We can group error metrics into two major categories:

📊 1. For Regression (Predicting Numbers)

Metric	Measures	Goal
MAE	Mean Absolute Error	Lower is better
MSE	Mean Squared Error	Lower is better
RMSE	Root Mean Squared Error	Lower is better
R^2 Score	Coefficient of Determination (0–1)	Closer to 1 is better

MAE (Mean Absolute Error): The average of the **absolute differences** between the actual and predicted values.

It tells you how far off your predictions are **on average**.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- y_i = actual value
- $\hat{y}_i$ = predicted value

The average of the **squared differences** between predicted and actual values.
Larger errors are **penalized more**.

📌 Formula:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

3. RMSE (Root Mean Squared Error)

Definition:

Square root of the MSE — it brings the error **back to the original units**.

📌 Formula:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

1. Example dataset

Suppose we are predicting house prices.

Actual price	Predicted price	Error
50	40	+10
60	70	-10
80	65	+15
90	100	-10

(All values are in lakhs.)

Absolute means negative values become positive.

$$MAE = \frac{\sum |Actual - Predicted|}{n}$$

$$MAE = \frac{10 + 10 + 15 + 10}{4}$$

MAE = 11.25 lakhs

Meaning: On average, our predictions are off by ₹11.25 lakhs.

MSE squares each error and calculates their average.

$$MSE = \frac{\sum(Actual - Predicted)^2}{n}$$

Error	Squared error
10	100
-10	100
15	225
-10	100

$$MSE = \frac{100 + 100 + 225 + 100}{4}$$

MSE = 131.25

Meaning: Larger errors receive more penalty because errors are squared.

RMSE is simply the square root of MSE.

$$RMSE = \sqrt{MSE}$$

$$RMSE = \sqrt{131.25}$$

RMSE = 11.46 lakhs

Meaning: RMSE gives the error in the original unit (lakhs), making it easier to interpret than MSE.

✅ 4. R^2 Score (R-squared)

Definition: Tells you how well your predictions explain the variance in the actual data. Values range from **0 to 1**.

```python
r2 = r2_score(y_true, y_pred)
print("R² Score:", r2)
```

📌 **Interpretation:**

- 1 = perfect model
- 0 = model does no better than the mean
- Can even be negative if the model performs worse than a horizontal line!

It measures how much of the variation in the dependent variable (target variable) is explained by the independent variables (features) used in the model.

In simple words:

It tells us how well our model explains the changes in the target variable.

Example:

Suppose you are building a Machine Learning model to predict house prices.

Your dataset contains:

- House area (square feet)
- Number of bedrooms
- Location
- Age of the house
- House price (target variable)

Your model predicts house prices based on these features.

Suppose your R^2 Score is 0.85.

It means the model explains approximately 85% of the variation in house prices in the evaluated dataset.

The remaining 15% is unexplained by the model.

In machine learning, the R^2 score, also known as the coefficient of determination, is a metric used to evaluate the goodness of fit of a regression model. It quantifies how well the model's predictions align with the actual data, ranging from 0 to 1. A higher R^2 score indicates a better fit, meaning the model explains a larger proportion of the variance in the dependent variable.

2. For Classification (Predicting Categories)

Metric	What it Measures
Accuracy	% of correct predictions
Precision	Of predicted positives, how many were correct
Recall	Of actual positives, how many were found
F1 Score	Harmonic mean of precision and recall
ROC-AUC	Probability model ranks a random positive higher than a random negative

****

MULTICOLLINEARITY

Multicollinearity refers to high correlation between independent variables, making it difficult to determine the individual effect of each variable on the outcome. The Variance Inflation Factor (VIF) is a measure used to detect and quantify the severity of multicollinearity. A VIF greater than 5 indicates moderate multicollinearity

For multicollinearity & vif ,see below path

C:\Users\User\Downloads\Internshala\Machine Learning\Linear Regression\M5 project

1. Identifying Multicollinearity:

Correlation Matrix:

Examine the correlation between independent variables. High correlation (e.g., above 0.8 or 0.9) suggests potential multicollinearity.

Variance Inflation Factor (VIF):

VIF measures how much the variance of a coefficient is increased due to multicollinearity. A VIF greater than 5 or 10 typically indicates a significant problem.

2. Addressing Multicollinearity:

Remove Highly Correlated Variables:

- **Stepwise Regression:** Use stepwise regression to identify and remove variables with the highest VIF values.
- **Manual Selection:** Based on VIF values or domain knowledge, remove one of the correlated variables.

Combine Variables:

- **Principal Component Analysis (PCA):** Transform correlated variables into uncorrelated principal components, reducing dimensionality while retaining most information.

PCA (Principal Component Analysis) is a powerful **unsupervised machine learning technique** used for: **Dimensionality reduction**

PCA in Practice (Using Scikit-learn)

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
X_pca = pca.fit_transform(X_scaled)

# Check explained variance
print(pca.explained_variance_ratio_)
```

Combine Variables:

- **Principal Component Analysis (PCA):** Transform correlated variables into uncorrelated principal components, reducing dimensionality while retaining most information.
- **Create a New Variable:** Combine variables mathematically to create a single, more meaningful variable (e.g., combining salary and business income into total income).

Regularization Techniques:

- **Ridge Regression:** Penalizes large coefficients, reducing the impact of highly correlated variables.
- **Lasso Regression:** Similar to ridge regression, but can also set coefficients to zero, effectively removing variables from the model.

Increase Sample Size:

- More data can help stabilize relationships between variables and reduce the impact of multicollinearity.

3. Considerations:

Explain PCA with an example", you can say:

"Suppose I have 10 independent variables in my dataset. Some variables may contain overlapping information. I apply PCA to convert these 10 variables into fewer components.
First, I standardize the data. Then PCA calculates eigenvalues and eigenvectors to identify the directions with maximum variance.
I select components based on explained variance. For example, if the first three components explain 95% of the total variance, I can retain those three components instead of all 10 variables.
Finally, I use these components as input to my machine learning model."

Why PCA is Called Unsupervised?

PCA does not use target/output labels. It works only on input features and identifies patterns based on variance.

Why "Unsupervised"?

Because PCA does NOT use target/output column.

Example supervised data:

Study Hours	Sleep	Result
5	7	Pass

Here:

```
Result
```

is target variable.

PCA Ignores Target Column

PCA only looks at:

- patterns
- variance
- relationships between input features

It does NOT care about:

```
Pass / Fail
```

So PCA learns without labels.

That is why it is called:

```
Unsupervised Learning
```

PCA Workflow

1.Standardize Data
2. Calculate Covariance Matrix
3. Find Eigenvectors and Eigenvalues
4. Select Principal Components
5. Transform Data

Advantages of PCA

- Reduces dimensions
- Faster model training
- Removes noise
- Reduces overfitting

Applications of PCA

- Image Compression
- Data Visualization
- NLP
- Noise Reduction

Why do we standardize data before PCA?

PCA is sensitive to scaling:
PCA is variance-based. Features with larger scales have larger variance and dominate the principal components. Standardization ensures all features contribute equally by bringing them to the same scale

What is Eigenvector in PCA?

Eigenvector represents the direction of maximum variance in the data.

What is Eigenvalue in PCA?

Eigenvalue represents the amount of variance captured by an eigenvector.

Why are Eigenvalues Important?

PCA selects principal components with highest eigenvalues because they contain maximum information.

- eigenvector gives the direction of the new axis, and eigenvalue tells how important that direction is

3. How do we decide the number of components?

By checking cumulative explained variance. For example, if two components explain 97% of variance and our threshold is 95%, we select two components.

Let's implement the complete PCA workflow in Python, from importing libraries to training a Linear Regression model.
We'll use an employee salary prediction example.
Our objective is to reduce 3 independent variables into fewer principal components while retaining 95% of the variance.

Step 1: Import libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

Step 2: Create sample dataset

```python
data = {
    'Age': [25, 30, 35, 40, 45, 28, 32, 38, 42, 48],
    'Experience': [2, 5, 8, 12, 15, 3, 6, 10, 13, 18],
    'Previous_Salary': [20000, 35000, 50000, 65000, 80000,
                        25000, 40000, 55000, 70000, 90000],
    'Current_Salary': [30000, 45000, 60000, 80000, 100000,
                       35000, 50000, 70000, 85000, 110000]
}

df = pd.DataFrame(data)

print(df)
```

Step 3: Separate X and Y

```python
X = df[['Age', 'Experience', 'Previous_Salary']]

y = df['Current_Salary']
```

We don't include the target variable in PCA.

Step 4: Split training and testing data

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- 80% data for training.
- 20% data for testing.

Why split first?

To prevent data leakage. PCA must learn its components using training data only.

Step 5: Standardize the data

```python
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

Why do we standardize?

Because Age, Experience and Previous Salary have different numerical scales.

StandardScaler converts them into comparable scales.

Remember:

- `fit_transform()` learns and transforms training data.
- `transform()` applies the same scaling parameters to testing data.

Step 6: Apply PCA

First, let's check how much variance each component explains.

```python
pca = PCA()

X_train_pca = pca.fit_transform(X_train_scaled)

print(pca.explained_variance_ratio_)
```

Example output (actual values depend on the dataset):

```
[0.85, 0.12, 0.03]
```

This means:

- PC1 explains 85%.
- PC2 explains 12%.
- PC3 explains 3%.

Step 7: Calculate cumulative explained variance

```
print(pca.explained_variance_ratio_.cumsum())
```

Example output:

```
Plain text
[0.85, 0.97, 1.00]
```

Interpretation:

Components	Cumulative variance
PC1	85%
PC1 + PC2	97%
PC1 + PC2 + PC3	100%

If our target is 95%, we need 2 components.

Step 8: Automatically select components

Instead of manually specifying 2, we can tell PCA to retain 95% variance.

```
Python
pca = PCA(n_components=0.95)

X_train_pca = pca.fit_transform(X_train_scaled)

X_test_pca = pca.transform(X_test_scaled)

print(X_train_pca.shape)
print(X_test_pca.shape)
```

The output will have:

- 8 rows and 2 components for training.
- 2 rows and 2 components for testing.

Step 9: Train Linear Regression model

Now we use the PCA-transformed data to train our regression model.

```python
model = LinearRegression()

model.fit(X_train_pca, y_train)
```

Notice that we are using `X_train_pca` instead of the original `X_train`.

Why? Because PCA has converted the original variables into principal componen

Step 10: Make predictions

```python
y_pred = model.predict(X_test_pca)

print(y_pred)
```

Now we have predicted salaries for the testing data.

Step 11: Evaluate the model

```python
print("R2 Score:", r2_score(y_test, y_pred))

print("RMSE:", mean_squared_error(
    y_test, y_pred
) ** 0.5)
```

- R^2 tells us how much variation in salary is explained by the model.
- RMSE tells us how far predictions are from actual salaries, on average in terms of squared-error scale.

Compare this model with a Linear Regression model trained on all original features to see whether PCA actually improves test performance.

**********,*

FACTOR ANALYSIS

Factor Analysis is a statistical technique used to reduce a large number of observed variables into a smaller number of hidden variables called factors.

The main purpose is to identify hidden relationships among variables and simplify complex datasets.

Instead of studying many variables separately, Factor Analysis groups related variables together into factors.
Main Idea of Factor Analysis
Suppose a dataset contains many variables that are related to each other.

Factor Analysis assumes that these variables are influenced by some hidden common factors.

Observed Variables → Hidden Factors

It reduces complexity and helps us understand the structure of data.

3. Observed Variables

Observed variables are variables that we can measure directly.

Examples:
- Height
- Weight
- Salary
- Math Marks
- Physics Marks

4. Latent (Hidden) Variables

Latent variables are hidden variables that cannot be measured directly.
Examples:
- Intelligence
- Academic Ability
- Customer Satisfaction
- Personality

We estimate latent variables using observed variables.

5. Simple Example

Suppose a student gets:
- High Math Marks
- High Physics Marks
- High Chemistry Marks

These subjects are related.
Factor Analysis may conclude that all these variables are influenced by one hidden factor called: Academic Ability.
So instead of analyzing 3 variables separately, we analyze one hidden factor.

6. Goal of Factor Analysis

The main goals are:
- Reduce the number of variables
- Find hidden patterns
- Identify relationships among variables
- Simplify datasets

7. Steps in Factor Analysis

Step 1: Collect Data
Step 2: Standardize Variables
Step 3: Create Correlation Matrix
Step 4: Extract Factors

Step 5: Rotate Factors
Step 6: Interpret Factors
8. Step 1 – Correlation Matrix
Factor Analysis first checks correlations among variables.
If variables are highly correlated, they may belong to the same hidden factor.

Example:
Math, Physics and Chemistry marks may have high correlation.
9. Step 2 – Feature Extraction
Feature Extraction identifies important hidden factors from observed variables.

Methods used:
- Principal Component Analysis (PCA)
- Maximum Likelihood

The goal is to determine how many important factors exist in the data.
10. Role of PCA in Factor Analysis
PCA is often used during extraction to determine the number of important factors.

PCA finds directions with maximum variance.

Factors with high variance are selected because they contain more information.
11. Eigenvalues in Factor Analysis
Eigenvalue tells how much information/variance a factor explains.

Rule:
Higher Eigenvalue → More Important Factor

Usually factors with eigenvalue > 1 are selected.
12. Step 3 – Feature Rotation
After extraction, factors may be difficult to interpret.

Rotation rearranges factors to make them clearer and easier to understand.

Rotation improves interpretability.
16. Real-Life Example
Customer Survey:
- Product Quality
- Packaging
- Delivery Speed
- Staff Behavior
- Cleanliness

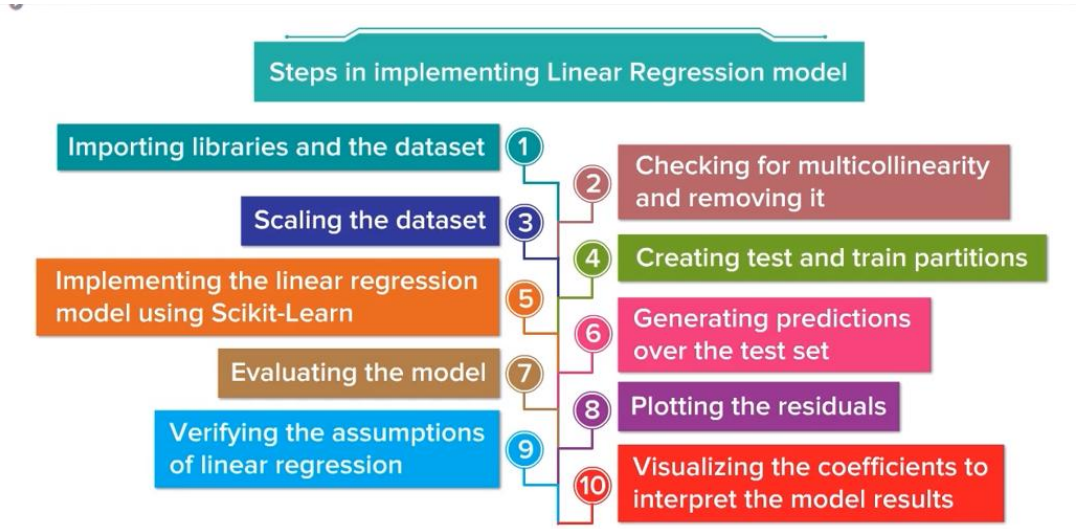
Factor Analysis may create:
- Product Experience
- Service Experience

20. Difference Between PCA and Factor Analysis

PCA:
- Focuses on variance
- Used for dimensionality reduction
- Creates principal components

Factor Analysis:
- Focuses on hidden factors
- Used for relationship modeling
- Creates latent variables

****



**********.

IMBALANCE DATASET

An **imbalanced dataset** is a dataset in which target variable are **not represented equally**. This is a common issue in **classification problems** and can seriously affect model performance.

⚖️ What Is Imbalanced Data?

Imagine a binary classification problem (e.g., fraud detection):

- **Class 0 (not fraud)**: 99,000 samples
- **Class 1 (fraud)**: 1,000 samples

Here, the data is **heavily imbalanced** — the model may just predict "not fraud" every time and still get 99% accuracy, but it fails at its actual goal.

🖍️ Problems Caused by Imbalanced Data

- **Biased models**: Models tend to favor the majority class.
- **Misleading accuracy**: High overall accuracy, but poor minority class performance.
- **Poor generalization**: Model fails in real-world us specially on rare but important classes.

🛠️ How to Handle Imbalanced Data

1. Resampling Techniques

a. **Undersampling** (reduce majority class)

- Randomly remove samples from the majority class.
- Pros: Fast, simple.
- Cons: May lose important information.

b. **Oversampling** (increase minority class)

- Duplicate or generate new samples from the minority class.

  ✅ Example: **SMOTE (Synthetic Minority Oversampling Technique)**
  It generates synthetic samples rather than just copying.

Under sampling is a technique used to handle imbalanced datasets by reducing the number of samples in the majority class so that it is more balanced with the minority class.

📊 Example:

Suppose your dataset has:

- **Class 0 (Majority):** 9,000 samples
- **Class 1 (Minority):** 1,000 samples

With **undersampling**, you randomly select 1,000 samples from Class 0 to match Class 1, resulting in:

- **Class 0:** 1,000 samples
- **Class 1:** 1,000 samples

Now the dataset is balanced (1:1 ratio).

python

```python
from imblearn.under_sampling import RandomUnderSampler
from collections import Counter

# X = features, y = target
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X, y)

print(f"Before: {Counter(y)}")
print(f"After: {Counter(y_resampled)}")
```

✅ Example with SMOTE

```python
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

sm = SMOTE(random_state=42)
X_resampled, y_resampled = sm.fit_resample(X, y)
```

✅ What Is **SMOTETomek**?

SMOTETomek is a **hybrid technique** that combines:

1. **SMOTE (Synthetic Minority Oversampling Technique)**
2. **Tomek Links (a data cleaning undersampling method)**

It's used to handle **imbalanced datasets** in machine learning by both **increasing** and **cleaning** data intelligently.

🎯 Why Use SMOTETomek?

Benefit	Explanation
✅ Balances Classes	SMOTE increases minority class size.
✅ Cleans Data	Tomek removes noisy or overlapping majority class samples.
✅ Improves Model Accuracy	Models train on more **representative and cleaner** data.
✅ Reduces Overfitting	Avoids simply duplicating samples or leaving noisy borders.

```python
from imblearn.combine import SMOTETomek
from sklearn.datasets import make_classification
from collections import Counter

# Generate imbalanced dataset
X, y = make_classification(n_samples=1000, n_features=10, weights=[0.9, 0.1], random_state=42)
print("Original:", Counter(y))

# Apply SMOTETomek
smote_tomek = SMOTETomek(random_state=42)
X_resampled, y_resampled = smote_tomek.fit_resample(X, y)

print("After SMOTETomek:", Counter(y_resampled))
```

Copy Edit

2. Class Weights

- Give more weight to the minority class during model training.
- Supported in many ML models (e.g., `class_weight='balanced'` in scikit-learn classifiers).

✅ Example in Scikit-learn: Using Class Weights

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(class_weight='balanced')
model.fit(X_train, y_train)
```

4. Evaluation Metrics

Avoid using just accuracy. Instead, use metrics like:

- **Precision**
- **Recall**
- **F1-score**
- **AUC-ROC (Area Under the ROC Curve)**
- **Confusion Matrix**

When to Use Precision in Machine Learning

You should focus on **precision** when you care more about the **correctness of positive predictions** than catching all actual positives.

Precision tells you:

👉 "Of all the instances the model predicted as **positive**, how many were **actually positive**?

Real-World Examples When to Use **Precision**

Precision focus on False Positives

Remember: Precision = Avoid incorrectly predicting positive.

It gives the percentage of false positive

📨 1. Spam Detection

- **False Positive (FP)**: A legitimate email is marked as spam.
- You want to **minimize these**, so users don't miss important emails.
- ✅ Focus on **precision**.

💉 2. Disease Screening (Preliminary Test)

- Imagine a rare disease with a first-level test.
- **False Positive**: Healthy person told they *might* have the disease → leads to stress and expensive follow-ups.
- In early screening, we want to be **precise**, so only real high-risk people are flagged.

Recall focuses on False Negatives (FN).

"Out of all the patients who actually have cancer, how many did the model correctly identify?"

From our example:

- Total patients who actually have cancer = 20.
- Model correctly identified 15.
- Model missed 5 patients.

Formula:

$$Recall = \frac{TP}{TP + FN}$$

Substitute the values:

$$Recall = \frac{15}{15 + 5}$$

$$= \frac{15}{20} = 0.75$$

Recall = 75%

Meaning: Out of 100 patients who actually have cancer, the model identifies 75.

Why does Recall focus on False Negatives?

Imagine a patient actually has cancer, but the model predicts that the patient is healthy.

This is dangerous because:

- Diagnosis may be delayed.
- Treatment may be delayed.
- The patient's condition may become worse.

Therefore, we want to reduce False Negatives.

Remember: Recall = Don't miss actual positive cases.

🛒 3. **Product Recommendation**

- Recommending the wrong product (FP) can annoy users or waste marketing budget.
- You want to **only recommend items** that the user is likely to buy or interact with.
- ✅ Use **precision** to make recommendations more relevant.

🚨 **4. Fraud Detection (in some cases)**

- If you act on **every predicted fraud**, false positives could mean **blocking innocent users**.
- You want to avoid accusing someone of fraud unless you're confident.
- ✅ Precision is important when the **cost of a false alarm is high**.

✅ **When to Use Recall in Machine Learning**

You should focus on **recall** when:

> **False negatives are more costly or dangerous** than false positives.

In other words, you care more about **not missing any actual positive cases**, even if it means making some false alarms.

📘 **Definition Recap:**

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **TP (True Positives)** = Correctly predicted positives
- **FN (False Negatives)** = Actual positives missed by the model

"Of all the actual **positive** cases, how many did the model **correctly identify**?"

🩺 1. **Medical Diagnosis (e.g., cancer detection)**

- **False Negative (FN)** = A sick person is told they're healthy.
- This could delay treatment and cost lives.
- You'd rather have **some false positives** (further testing) than **miss a real case**.
- ✅ Maximize **recall**.

🚨 2. **Criminal/Threat Detection**

- Predicting if someone is a threat based on behavior or biometric analysis.
- Missing an actual threat (FN) could lead to harm.
- You want to **catch all real threats**, even if it means some innocent people get flagged for further checks.
- ✅ Prioritize **recall**.

🧪 3. **Search Engines / Information Retrieval**

- Recall measures how many relevant documents were actually found.
- You want to **show all the useful results**, even if some irrelevant ones are included.
- ✅ Use **recall** when coverage is more important than precision.

🧬 4. **Rare Event Prediction (e.g., equipment failure, fraud)**

- You can't afford to **miss an actual failure**.
- Missing a true fault (FN) means a crash, downtime, or loss.
- ✅ Maximize **recall**, especially in **anomaly detection**.

⚠️ When Not to Use Recall Alone

Avoid using **recall by itself** when:

- **False positives are expensive** (e.g., spam filters, criminal accusations).
- You need a balance between catching all positives and not over-predicting.

In such cases, combine it with **precision** using the **F1 Score**.

✅ What is the **F1 Score** in Machine Learning?

The **F1 Score** is the **harmonic mean** of **precision** and **recall**.

It gives you a **single performance metric** that balances both false positives and false negatives.

🧠 When to Use the F1 Score:

> Use **F1 Score** when you **need a balance between precision and recall**, especially when you have **imbalanced datasets**.

🎯 Real-World Examples

🔴 1. **Fraud Detection**

- Fraud is rare (imbalanced dataset).
- You want to **catch fraud** (high recall), but also **not flag too many legitimate users** (high precision).
- ✅ Use **F1 Score** to strike a balance.

📘 What is Log Loss?

Log Loss (also called **logarithmic loss** or **binary cross-entropy**) is an **evaluation metric** used to assess the **quality of probability predictions** in **classification models.**

Unlike accuracy, which just checks if predictions are correct, **log loss measures how close the predicted probabilities are to the actual class labels.**

✅ When to Use Log Loss:

Use Log Loss When...	Reason
Your model predicts **probabilities**	Log loss evaluates probability quality
You care about **confidence** in predictions	Encourages better calibration
You're using models like **logistic regression**, **neural networks**, **XGBoost**, etc.	These models output class probabilities
Your dataset is **imbalanced** or you need **ranking of predictions**	Log loss considers more than just correct/incorrect

✅ What is a **Coefficient Plot** in Machine Learning?

A **coefficient plot** is a **visual representation of the feature coefficients (weights)** from linear models like:

- **Logistic Regression**
- **Linear Regression**
- **Lasso / Ridge Regression**

It helps you understand:

- **Which features have the biggest impact**
- **Direction** of influence (positive or negative)
- **Relative importance** of each feature

Watch internshala module 7 video for detailed explanation

Coefficient Plot

[43]:
```
# arranging the data
c = classifier.coef_.reshape(-1)
x = X.columns

coeff_plot = pd.DataFrame( {
                            'coefficients': c,
                            'variable' : x
                            })

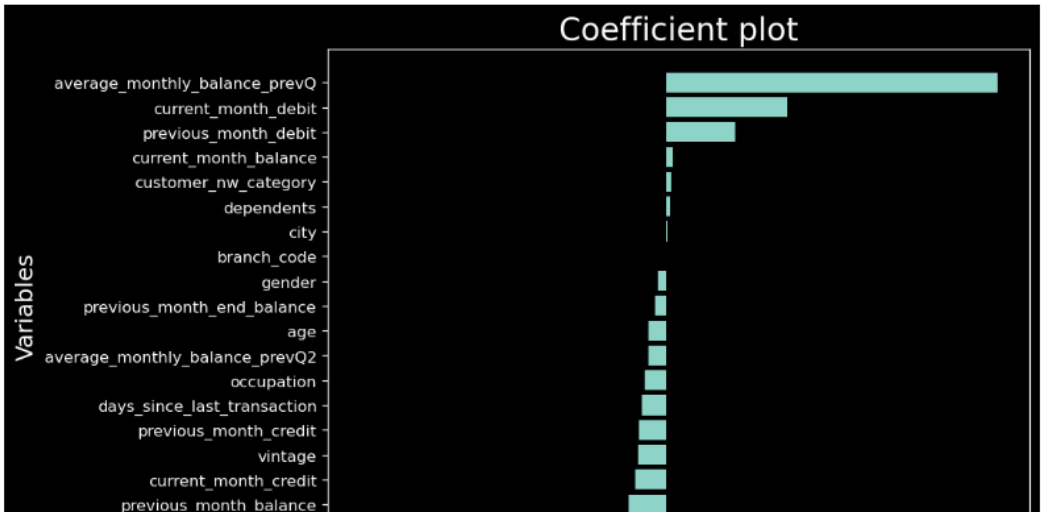
# sorting the values
coeff_plot = coeff_plot.sort_values(by = 'coefficients')
coeff_plot.head()
```

Out[43]:

	coefficients	variable
9	-2.300100	current_balance
13	-0.603295	current_month_credit
0	-0.175263	vintage
10	-0.169757	previous_month_end_balance

```
plt.figure(figsize=(8, 6), dpi=120)
plt.barh( coeff_plot['variable'] , coeff_plot['coefficients'] )
plt.xlabel( "Coefficient Magnitude", fontsize = 15)
plt.ylabel('Variables', fontsize = 15)
plt.title('Coefficient plot', fontsize = 20)
```

```
Text(0.5, 1.0, 'Coefficient plot')
```



✅ Parametric Model – Definition:

A **parametric model** is a type of machine learning model that **assumes a fixed form or structure** (like a formula) and learns a **set number of parameters** from the data.

> 📄 **In simple terms:**
> It's like solving a problem using a fixed equation. The model makes assumptions about how the data behaves.

🧠 **Examples:**

- Linear Regression
- Logistic Regression
- Naive Bayes

A parametric model is that model which has finite and fixed set of parameters to learn from

–

✅ Non-Parametric Model – Definition:

A **non-parametric model** does **not assume any fixed structure.** Instead, it **learns patterns directly from the data**, and its complexity can grow as it sees more data.

> 📄 **In simple terms:**
> It's like learning by remembering past experiences. The model is more flexible and can adapt to the data's shape.

🧠 **Examples:**

- k-Nearest Neighbors (k-NN)
- Decision Trees
- Random Forest
- Support Vector Machine (with non-linear kernels)

A non-parametric model is that model that keeps on learning from the data until there is nothing left to learn

- # DEBUGGIN AN ERROR IN PYTHON CODE

✅ 1. Understand the Error Message

When Python throws an error, it gives you a **traceback** that tells you:

- **Type of error** (e.g., `TypeError`, `NameError`, `ValueError`)
- **Line number** where the error occurred
- **File name** and function (if applicable)

🧠 **Tip:** Always start reading from the **bottom line** of the traceback — that's the actual error.

✅ 3. Use `try` / `except` Blocks to Catch Errors

Catch and handle known exceptions to avoid crashing.

```python
try:
    result = a / b
except ZeroDivisionError:
    print("You can't divide by zero!")
```

✅ 6. Check for Common Mistakes

- Indentation errors (`IndentationError`)
- Misspelled variable names (`NameError`)
- Wrong data types (`TypeError`)
- Accessing lists/dicts out of range (`IndexError` , `KeyError`)
- Wrong function arguments (`TypeError: missing 1 required positional argument`)

****

Feature engineering:

Rescaling Technique:

Scaling is important in machine learning to bring all features to the same scale so that the model does not give more importance to features with higher magnitude. (useful for distance-based algorithm)

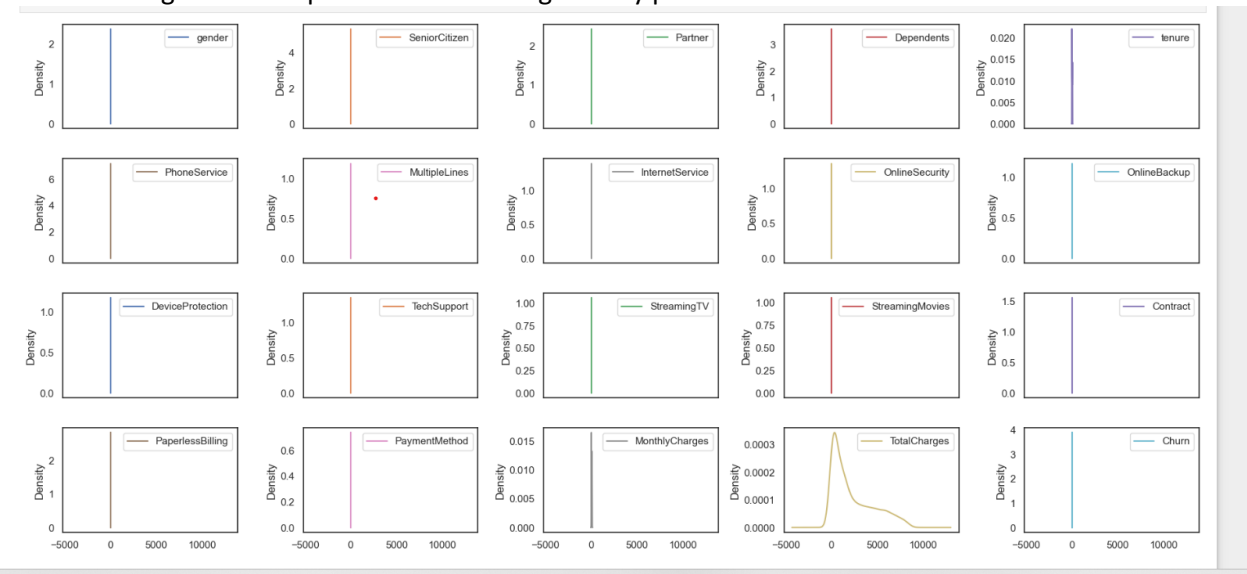
Scaling is not important for Decision trees, random forest & gradient boosting algorithm because they split data based on feature thresholds rather than distance or magnitude.

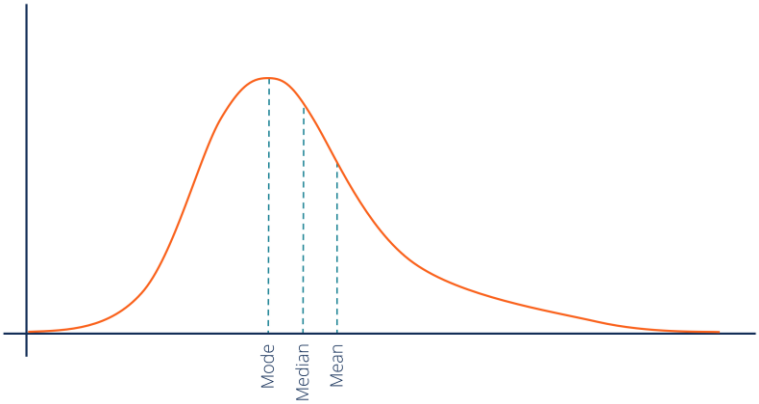
1.Standardsiaton (Z score scaling): Standardization rescales data so that:
Mean = 0 and Standard Deviation = 1
Use it when data is normally distributed or if the data has outliers. We use it in **most ML problems**, especially with **linear, distance-based, and PCA-based models**.
2.Normalization (MIN-MAX scaling): **Normalization** rescales values into a **fixed range (usually 0–1).**
Use normalization when data has no extreme outliers or outliers are already handled before

Skewness:

Skewness describes the asymmetry of a data distribution. It shows whether the values in a dataset are leaning more toward the left or the right side of the distribution.
If the data is perfectly balanced, it has zero skewness (normal distribution
skewness analysis should only be applied to continuous numerical variables. Applying KDE on label

encoded categorical data produces misleading density plots because the numeric order is artificial.

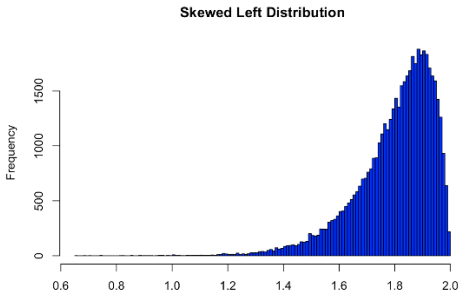




Tail is longer on the right side
Most values are small, but a few very large values exist
Example:

- Income distribution
- House prices

Key relation:
Mean > Median > Mode



Tail is longer on the left side
Most values are large, but a few very small values exist

Example:

- Easy exam scores (many students score high)

Key relation:

Mean < Median < Mode

Skewness can be treated using transformations like square root and cube root. Square root transformation is used for moderately skewed data, while cube root transformation is useful for highly skewed data and can also handle negative values. These transformations compress large values and help make the distribution more normal.

Removing skewness ¶

```
9]: #treating skewness via squareroot method and cube root method
    df.skew()
    for col in df.skew().index:
        if col in df.describe().columns:
            if df[col].skew()>0.55:
                df[col]=np.sqrt(df[col])
            if df[col].skew()<-0.55:
                df[col]=np.cbrt(df[col])
```

```
0]: df.skew()
```

```
0]: label                   -2.271543
    aon                      0.307552
    daily_decr30             1.129445
```

We use .index because df.skew() returns a Series containing column names and their skewness values. We only need the column names to iterate through each numerical feature, so we extract them using .index.

```
: df.skew().index
```

```
: Index(['label', 'aon', 'daily_decr30', 'daily_decr90', 'last_rech_amt_ma',
         'cnt_ma_rech30', 'sumamnt_ma_rech30', 'medianamnt_ma_rech30',
         'cnt_ma_rech90', 'fr_ma_rech90', 'sumamnt_ma_rech90',
         'medianamnt_ma_rech90', 'medianmarechprebal90', 'cnt_da_rech30',
         'cnt_loans30', 'amnt_loans30', 'maxamnt_loans30', 'medianamnt_loans30',
         'cnt_loans90', 'amnt_loans90', 'maxamnt_loans90', 'medianamnt_loans90',
         'payback30', 'payback90', 'rental30', 'rental90'],
        dtype='object')
```

```
: df.skew()
```

```
: label                   -2.271543
  aon                      0.307552
  daily_decr30             1.129445
  daily_decr90             1.234495
  last_rech_amt_ma         0.802090
  cnt_ma_rech30            0.361829
  sumamnt_ma_rech30        0.815695
```

- we should not apply skewness transformations to the target column in a classification problem because the target represents class labels (categories), not continuous numerical values.

For example:

0 = No

1 = Yes

Applying square root or cube root would change these class labels and their meaning.

We need to preserve the original class values. Skewness treatment is generally applied to numerical independent features when required.

```
for col in df.drop(columns=['label']).skew().index:

    if df[col].skew() > 0.55:
        df[col] = np.sqrt(df[col])

    elif df[col].skew() < -0.55:
        df[col] = np.cbrt(df[col])
```

➢ Explain logistic regression:

Logistic Regression is a supervised machine learning algorithm used for classification problems. It learns patterns from historical data and predicts the probability of a data point belonging to a particular class. It uses the Sigmoid activation function to convert the output into a probability between 0 and 1, and based on a threshold value of 0.5, it predicts the final class.

Activation function in logistics regression.:Sigmoid function

- **Cross validation score**
- **Optuna**

Ensemble Learning Technique

Instead of using a single model, we use a group of model & based o prediction of each model we make the final decision.

Example:

Model 1 → 0

Model 2 → 1

Model 3 → 0

Final prediction = 0 (because most models predicted 0).

How do we combine models?

Classification: We use majority voting. The class predicted by most models is taken as the final prediction.

Regression: We take the average of all predictions.

Advantages of Ensemble Technique

1. Better prediction: When multiple models work together, they can capture different patterns in the data that a single model may miss.
2. Reduce overfitting: As ensemble techniques combine predictions from multiple models, they can reduce the overfitting effect of individual models.
3. Improves performance: Combining multiple models can improve the overall prediction performance.

Disadvantages

1. Increases complexity: We are building several models and combining their predictions. This increases complexity.
2. Not interpretable as a whole: It becomes difficult to understand how all models together make the final prediction.
3. Increases time complexity and computational requirements: Training multiple models requires more time and resources.

Bagging (Bootstrap Aggregating)

In bootstrap sampling, each model is given a random sample from the original dataset with replacement.
Let's understand this with an example.
Suppose we have 26 unique observations representing A to Z.
The most straightforward way to build multiple models is to divide the data equally among them.
For example:

- Model 1 → A, B, C, D
- Model 2 → E, F, G, H
- Model 3 → I, J, K, L

What is the problem with this approach?
Each model is getting only a very small portion of the actual dataset. As a result, each model learns only a small portion of the underlying pattern.
To overcome this issue, we use the concept of Bootstrap Sampling.
Suppose we have 26 observations. To create the first sample, we randomly select 26 values from the original dataset.
For example:

- Sample 1 → A, B, C, A, D, E, F...
- Sample 2 → B, C, D, E, A, B, G...
- Sample 3 → Z, Y, X, A, B, C, Z...

Every time we select a value, the selection is random.
Important points:

- Some values may repeat.
- Some values may not appear even once.
- Each sample contains 26 observations.
- All samples are created independently from the original dataset.

By doing this, each model gets a slightly different but full-sized sample of data.
Bootstrap sampling is the core idea of the Bagging technique.

➢ Random Forest

Random Forest is a supervised machine learning algorithm used for classification and regression problems.

In classification, it works by creating multiple decision trees and combining their predictions to make a final prediction. The final class is chosen based on majority voting among all the trees.
A single Decision Tree can sometimes memorize the training data and perform poorly on new data (overfitting).
Random Forest reduces this problem by combining multiple trees, making predictions more accurate and stable.

What is Bootstrap Sampling?

Random Forest uses random samples of the training data (with replacement) to train each decision tree. This technique is called bootstrap sampling.

What is Majority Voting?

In classification, each tree predicts a class. The class that receives the highest number of votes becomes the final prediction

How Random Forest works

Suppose we want to create a Random Forest containing 10 Decision Trees.

step 1: First, we create 10 bootstrap samples from the original training data.
Step 2: We build one Decision Tree on each bootstrap sample.
For example:

- Bootstrap Sample 1 → Decision Tree 1
- Bootstrap Sample 2 → Decision Tree 2
- Bootstrap Sample 3 → Decision Tree 3
- Similarly, we create 10 Decision Trees.

Step 3: Each Decision Tree generates its own prediction based on the data it received.
Step 4: Lastly, we combine the predictions of all Decision Trees to get the final output.
This is why it is called a Forest, because it consists of multiple Decision Trees.
It is called Random Forest because it uses:

- Random sampling of data points.
- Random selection of features while building trees.

Final Prediction

- In Classification, we use majority voting.
- In Regression, we take the average of all predictions.

In Python, we specify the number of trees using:

```python
RandomForestClassifier(n_estimators=10)
```

Random Forest Hyperparameters (Interview Explanation)

1. n_estimators
Specifies the number of decision trees in the Random Forest.
However, increasing the number of trees also increases training time and computational cost.
Example: n_estimators = 100 means the model builds 100 decision trees.

---

2. max_depth
Controls the maximum depth of each decision tree.
A very deep tree can learn noise from the training data and lead to overfitting.
A shallow tree may miss important patterns and lead to underfitting.
Purpose: Helps balance model complexity.

---

3. min_samples_split
Defines the minimum number of samples required to split a node.
Higher values prevent the tree from creating too many branches.
Helps reduce overfitting by making the tree less complex.
Example: If min_samples_split = 10, a node must have at least 10 records before it can split.

---

4. min_samples_leaf
Specifies the minimum number of samples that must be present in a leaf node.
Example: If min_samples_leaf = 5, every leaf must contain at least 5 records.

---

5. max_features
Determines the number of features randomly selected at each split.
Introducing randomness creates diverse trees.

Diverse trees improve overall model performance and reduce overfitting.
Example: If there are 10 features and max_features = 3, only 3 features are considered at each split.

Random Forest Regressor is a supervised learning algorithm used for predicting continuous values. It builds multiple decision trees using random subsets of data and features. Each tree predicts a value, and the final prediction is obtained by taking the average of all tree predictions. This approach improves accuracy and reduces overfitting compared to a single decision tree.

Boosting – Ensemble Learning Technique

Boosting is an ensemble learning technique in which multiple models are trained sequentially (one after another).
Each new model tries to correct the mistakes made by the previous model.
The main idea of Boosting is to improve the overall performance by learning from previous errors.

How Boosting works

Suppose we want to build a model to predict whether a customer will purchase a product.
Step 1: First, we train the initial model using the training dataset.
Step 2: The model makes predictions. Some predictions may be correct, and some may be incorrect.
Step 3: The next model focuses more on the observations that were incorrectly predicted by the previous model.
Step 4: This process continues. Each new model tries to improve the mistakes made by the previous models.
Step 5: Finally, we combine the predictions of all models to get the final output.

Difference between Boosting Models

The three popular boosting algorithms are AdaBoost, Gradient Boosting and XGBoost.

1. AdaBoost (Adaptive Boosting)

- Gives more weight to incorrectly predicted observations.
- The next model focuses more on the previous model's mistakes.
- Combines predictions using weighted voting (classification) or weighted sums (regression).

Layman example:
Imagine a teacher conducting a test. Students who answer incorrectly get more attention in the next class so they can improve.
How it works: Focuses on wrong predictions by increasing their weights.

2. Gradient Boosting

- Builds models sequentially.
- Calculates the errors (residuals) made by the previous model.
- The next model tries to predict and correct those errors.
- Combines all model predictions to reduce overall error.

Layman example:
Suppose you estimate a house price as ₹40 lakh, but the actual price is ₹50 lakh. The next model learns the ₹10 lakh error and tries to correct the prediction.
How it works: Learns from previous errors using gradient-based optimization.

3. XGBoost (Extreme Gradient Boosting)

- An advanced and optimized version of Gradient Boosting.
- Uses regularization to control overfitting.
- Designed for faster and more efficient training.
- Supports parallel processing for certain computations.

Layman example:

Suppose Gradient Boosting is predicting house prices. XGBoost does a similar task but also uses additional techniques to control overfitting and improve training efficiency.
How it works: Gradient Boosting + Regularization + Performance Optimization.

Common example dataset

Problem: Predict whether a customer will purchase a product. Age and Income are input features (X). Purchased is the target (y): 0 = No, 1 = Yes.

Customer	Age	Income (₹000)	Purchased
A	22	20	0
B	25	25	0
C	35	50	1
D	45	60	1
E	28	30	0
F	40	55	1

Prepare the data

```
import pandas as pd

df = pd.DataFrame({
    'Age': [22, 25, 35, 45, 28, 40],
    'Income': [20, 25, 50, 60, 30, 55],
    'Purchased': [0, 0, 1, 1, 0, 1]
})

X = df[['Age', 'Income']]
y = df['Purchased']
```

3. AdaBoost (Adaptive Boosting)

Main idea: AdaBoost increases the weights of observations that the previous learner predicted incorrectly. The next learner pays more attention to those observations.

Layman example: A teacher notices that students B and E answered incorrectly, so the teacher gives them extra attention in the next practice session.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=5,
    learning_rate=1.0,
    random_state=42
)

ada.fit(X, y)
y_pred_ada = ada.predict(X)
y_prob_ada = ada.predict_proba(X)
```

Explanation:

- estimator=DecisionTreeClassifier(max_depth=1): Uses a small Decision Tree called a Decision Stump. It has at most one split level.
- n_estimators=5: Builds 5 weak learners sequentially.
- learning_rate=1.0: Controls each learner's contribution.
- random_state=42: Helps make results reproducible.
- fit(X, y): Trains the model.
- predict(X): Returns the predicted class (0 or 1).
- predict_proba(X): Returns the probability for each class.

Note: AdaBoost can use other base estimators too; shallow Decision Trees are commonly used.

4. Gradient Boosting

Main idea: Gradient Boosting builds trees sequentially. Each new tree follows the negative gradient of the loss function to improve the existing model. For squared-error regression, this is closely related to learning residuals.

Layman example: You estimate a house price as ₹40 lakh, but the actual price is ₹50 lakh. The next tree learns how to reduce the ₹10 lakh error. Further trees keep refining the prediction.

```
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(
    n_estimators=5,
    learning_rate=0.1,
    max_depth=1,
    random_state=42
)

gb.fit(X, y)
y_pred_gb = gb.predict(X)
y_prob_gb = gb.predict_proba(X)
```

- n_estimators=5: Builds 5 boosting trees.
- learning_rate=0.1: Controls how much each new tree contributes.
- max_depth=1: Each tree has at most one split level.
- predict(X): Returns the predicted class.
- predict_proba(X): Returns probabilities for class 0 and class 1.

Important: GradientBoostingClassifier predicts class probabilities. GradientBoostingRegressor predicts continuous numerical values.

5. XGBoost (Extreme Gradient Boosting)

Main idea: XGBoost is an optimized gradient-boosting implementation. It uses gradients and second-order information (Hessians), regularization, and engineering optimizations to improve efficiency and control model complexity.

Layman example: Like Gradient Boosting, it improves predictions step by step. XGBoost also adds controls to discourage overly complex trees and includes optimizations to make training more efficient.

```
pip install xgboost
from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=5,
    learning_rate=0.1,
    max_depth=1,
    eval_metric='logloss',
    random_state=42
)

xgb.fit(X, y)
y_pred_xgb = xgb.predict(X)
y_prob_xgb = xgb.predict_proba(X)
```

- n_estimators=5: Builds 5 boosting trees.
- learning_rate=0.1: Controls each tree's contribution.
- max_depth=1: Limits the depth of each tree.
- eval_metric=logloss: Uses log loss as the classification evaluation metric.
- predict(X): Returns the predicted class; predict_proba(X) returns class probabilities.

XGBoost can also be used for regression with XGBRegressor.

Underfitting & overfitting

Underfitting occurs when the model is too simple and cannot capture the pattern in the data, resulting in high training and testing error. Overfitting occurs when the model learns the training data too well, including noise, leading to low training error but high testing error.
If training & testing both accuracy is low-underfitting
If training accuracy is very high but testing accuracy is low-over fitting

Cross validation

ROC-AUC curve is used to evaluate binary classification models. It shows the tradeoff between true positive rate and false positive rate at different thresholds. The AUC score measures the model's ability to distinguish between classes, where higher values indicate better performance.
ROC-AUC curve is used when the dataset is balanced and measures the tradeoff between true positive rate and false positive rate. Precision–Recall curve is preferred for imbalanced datasets because it focuses on correctly identifying the minority class

Why Precision–Recall is Better for Imbalanced Data

Example:

Dataset:

Class	Count
Fraud	10
Not Fraud	1000

If the model predicts **all as not fraud**:

Accuracy:

```
Plain text
1000 / 1010 = 99%
```

Accuracy looks great, but the model **detects zero fraud**.

Precision–Recall curve exposes this problem.

Simple Comparison

Metric	When to Use
ROC Curve	Balanced datasets
Precision–Recall Curve	Imbalanced datasets

What is Z-score?
The Z-score measures how far a data point is from the mean in terms of standard deviations.
Uses:
Detect outliers
Standardize data

Why removing highly correlated features are considered a good practice?

When two features are highly correlated, they may provide similar information to the model, which may cause overfitting. If there are highly correlated features in the dataset then they unnecessarily increase the dimensionality of the feature space and sometimes create the problem of the curse of dimensionality. If the dimensionality of the feature space is high then the model training may take more time than expected, it will increase the complexity of the model and chances of error. This somehow also helps us to achieve data compression as the features have been removed without much loss of data.

**********.

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) is a supervised machine learning algorithm used for:
Classification – Predicting the class of new observations.

Dimensionality Reduction – Reducing the number of features while preserving class information.

Why Do We Need LDA?

Suppose we have student data:

Study Hours	Attendance	Result
8	90	Pass
7	85	Pass
2	40	Fail
3	50	Fail

LDA tries to find the best line that separates **Pass** and **Fail** students.

Goal of LDA

LDA aims to:

✅ Maximize the distance between different classes

✅ Minimize the distance within the same class

In simple terms:

Feature	LDA	PCA
Type	Supervised	Unsupervised
Uses Target Variable	Yes	No
Goal	Maximize class separation	Maximize variance
Mainly Used For	Classification	Dimensionality Reduction

Why is LDA better for classification?

Because LDA uses the target variable (class labels) and tries to maximize the distance between classes while minimizing the spread within each class. Therefore, it often provides better class separation than PCA for classification problems.

- PCA and LDA are dimensionality reduction techniques. PCA is an unsupervised method that reduces dimensions by retaining maximum variance in the data without considering class labels. LDA is a supervised method that uses class labels and reduces dimensions by maximizing the separation between classes. Therefore, PCA is mainly used for data compression and visualization, while LDA is commonly used as a preprocessing step for classification problems.

**********.**********

TIMESERIES

In time series analysis, we set the date or month column as the index because time is the primary dimension of the data. This allows Pandas to perform time-based operations such as filtering by date, plotting trends, rolling statistics, resampling, and forecasting more efficiently. Without setting the date column as the index, Pandas treats it as a regular column and cannot fully utilize its time-series functionality.

[12]:
```
df['Month']=pd.to_datetime(df['Month'])
df
```

[12]:

	Month	#Passengers
0	1949-01-01	112
1	1949-02-01	118
2	1949-03-01	132
3	1949-04-01	129
4	1949-05-01	121
...	...	...
139	1960-08-01	606

[13]:
```
df = df.set_index('Month')
df.head()
```

[13]:

Month	#Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

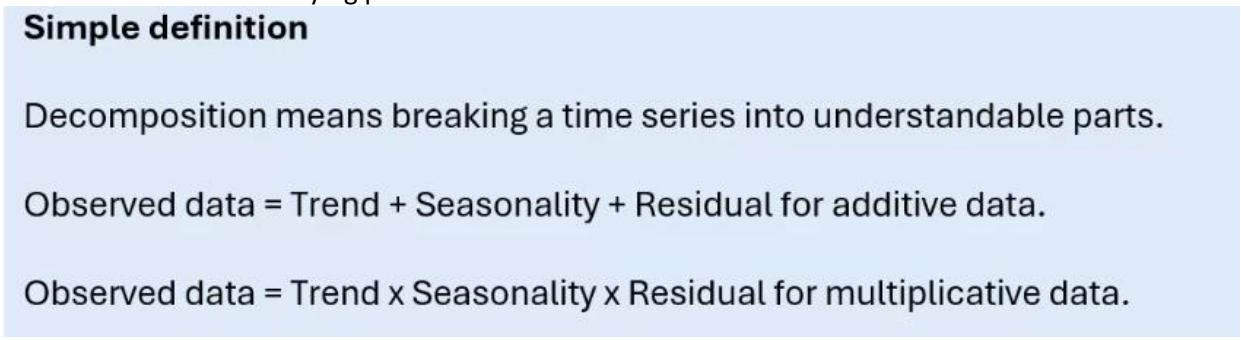
Rolling mean & rolling std. deviation

A rolling mean (moving average) calculates the average over a fixed window.
Example (window = 3):

We calculate rolling mean and rolling standard deviation to check whether a time series is stationary. A stationary series has a constant mean and variance over time. If the rolling mean and rolling standard deviation remain relatively constant, the series is likely stationary. If they change significantly, the series may contain trends or changing variance and needs transformation before applying forecasting models like ARIMA.
******.**********

DECOMPOSITION

Decomposition is a technique used in time series analysis to break data into smaller components so we can understand the underlying patterns.



- Additive & multiplicative decomposition

Time series decomposition is used to separate a time series into trend, seasonality, and residual components. In additive decomposition, the components are added together and are suitable when seasonal variations remain constant over time. In multiplicative decomposition, the components are multiplied and are suitable when seasonal variations increase or decrease with the trend.
**********.**********

Autocorrelation Function (ACF)

ACF measures the correlation between a time series and its past values (lags).

In simple words:
ACF tells us how strongly today's value is related to previous values such as yesterday, 2 days ago, 3 days ago, etc.

&&&

The interviewer may ask:

> "Did you apply SMOTE before or after train-test split?"

Correct approach:

Python

```python
X_train, X_test, y_train, y_test = train_test_split(...)

smote = SMOTE()
X_train_smote, y_train_smote = smote.fit_resample(X_train,y_train)
```

Wrong approach:

Python

```python
smote.fit_resample(X,y)
train_test_split(...)
```

Why?

Because synthetic data leaks information from test d into training data.

Gradient Boosting?
Gradient Boosting is an ensemble machine learning algorithm that combines multiple weak learners (usually Decision Trees) to create a strong predictive model. It builds trees sequentially, where each new tree tries to correct the errors made by the previous trees.
The algorithm uses Gradient Descent optimization to minimize the loss function by reducing prediction errors step by step.
How did you implement Gradient Boosting in code?", you can show/explain it like this:

```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
gb_model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)

# Train model
gb_model.fit(X_train, y_train)

# Predictions
y_pred = gb_model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Hyperparameter Tuning using GridSearchCV

Python

```python
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import GradientBoostingClassifier

params = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 5]
}

gb = GradientBoostingClassifier(random_state=42)

grid = GridSearchCV(
    estimator=gb,
    param_grid=params,
    cv=5,
    scoring='roc_auc'
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
```

**********8.

Assessment question:

You are writing a data cleaning script for an e-commerce platform. The goal is to remove any metadata keys from an order dictionary that are not part of the core schema (id, user_id, amount).

Why does the following code crash, and how should it be structured to work correctly?

2]:
```python
def clean_order_data(order: dict):
    allowed_keys = {'id', 'user_id', 'amount'}

    for key in order:
        if key not in allowed_keys:
            del order[key]

    return order


# Test:
clean_order_data({
    'id': 1,
    'amount': 25.0,
    'user_id': 'u1',
    'promo': 'SAVE10'
})
```

answer : above code crashes because it deletes keys from a dictionary while iterating over it. Python does not allow changing the size of a dictionary during iteration and raises:RuntimeError: dictionary changed size during iteration

Step 1: Function Call

```python
order = {
    'id': 1,
    'amount': 25.0,
    'user_id': 'u1',
    'promo': 'SAVE10'
}
```

Dictionary contains 4 keys:

```
id
amount
user_id
promo
```

Iteration 4

```python
key = 'promo'
```

Check:

```python
'promo' not in allowed_keys
```

Result:

```python
True
```

Execute:

```python
del order[key]
```

which becomes:

```python
del order['promo']
```

Problem Happens Here

Python started the loop expecting a dictionary with:
4 keys
Now suddenly the dictionary has:
3 keys
The size changed while Python was still looping.
Python detects:
Dictionary size changed during iteration
and immediately raises:

list(order.keys()) creates a separate copy of the dictionary keys. The loop iterates over that list instead of the original dictionary. Therefore, even if keys are added or deleted from the dictionary during the loop, the list being iterated over remains unchanged, so Python does not raise a RuntimeError.
RuntimeError:
dictionary changed size during iteration

```
#What will be the output of the following code? Explain why.
def add_item(item, lst=[]):
    lst.append(item)
    return lst

print(add_item(1))
print(add_item(2))
```

```
[1]
[1, 2]
```

The list is created only once because it is a default parameter. Python evaluates default parameters at function definition time, not at function call time. Therefore, the same list is reused across multiple calls, and every new value gets appended to that same list.

```
#this will not append every item to same list
def add_item(item, lst=None):

    if lst is None:
        lst = []

    lst.append(item)

    return lst
print(add_item(1))
print(add_item(2))
print(add_item(8))
print(add_item(1, [10,20]))
```

```
[1]
[2]
[8]
[10, 20, 1]
```

2. Dictionary Changed During Iteration

Question

Python

```python
d = {'a':1,'b':2,'c':3}

for k in d:
    del d[k]
```

Error

```
RuntimeError:
dictionary changed size during iteration
```

Fix

Python

```python
for k in list(d.keys()):
    del d[k]
```

---

4. Class Variable Sharing

Question

Python

```python
class Student:
    history = []

s1 = Student()
s2 = Student()

s1.history.append("Login")

print(s2.history)
```

Output

Python

```
['Login']
```

Why?

Class variables are shared among all objects.

7. String Immutability

Question

```python
s = "hello"

s[0] = "H"
```

Error

```
TypeError
```

Why?

Strings are immutable.

16. Default Argument Evaluation

Question

```python
def test(x=[]):
    x.append(1)
    return x

print(test())
print(test())
```

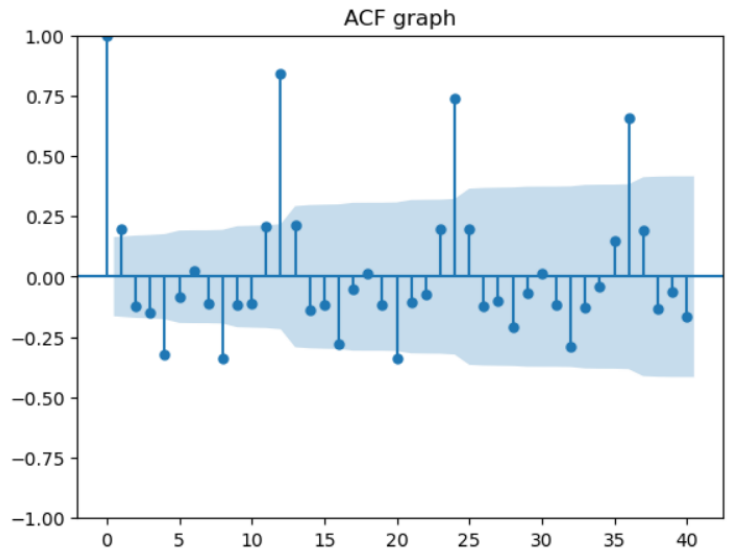
Output

```python
[1]
[1, 1]
```

Same mutable default argument issue.

```python
from statsmodels.graphics.tsaplots import plot_acf
plt.figure(figsize=(10,4))
plot_acf(df['log_first_diff' ].dropna(), lags = 40)
plt.title("ACF graph")
plt.show()
```

```
<Figure size 1000x400 with 0 Axes>
```

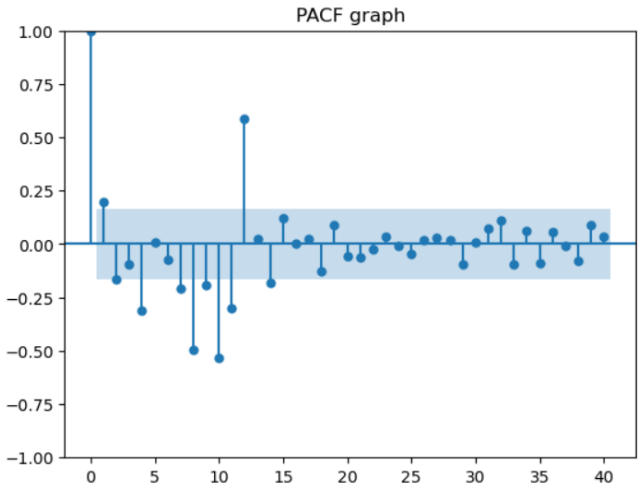


The ACF plot indicates strong yearly seasonality because significant spikes appear at lags 12, 24, and 36, meaning the pattern repeats every 12 periods. The plot suggests:

✅ Seasonality exists
✅ Data is not completely random
✅ Seasonal models like:
SARIMA
Holt-Winters
Seasonal Decomposition
**********.***
PACF (Partial Autocorrelation Function)

```
# from statsmodels.tsa.stattools import acf, pacf
from statsmodels.graphics.tsaplots import plot_pacf
plt.figure(figsize=(10,4))
plot_pacf(df['log_first_diff'].dropna(), lags = 40)
plt.title("PACF graph")
plt.show()
```

```
<Figure size 1000x400 with 0 Axes>
```



**********.

ARIMA